BASS as a Surrogate Model for Bayesian Optimization

Adrian Tame Jacobo

INSERT DATE HERE WHEN DONE

Contents

1 Stochastic Processes **5**
- 1.1 General Definitions . . . 6
- 1.2 Markov Chains . . . 7
- 1.3 Gaussian Processes . . . 13
- 1.4 Gaussian Process Regression . . . 17
  - 1.4.1 Preamble to Gaussian Process Regression: A Rehash on Regression Analysis . . . 20
  - 1.4.2 Theory of Gaussian Process Regression (from the function space view) . . . 22
- 1.5 Summary and Outlook . . . 24

2 Bayesian Optimization **25**
- 2.1 Introduction . . . 25
- 2.2 Components of BO . . . 27
- 2.3 Bayesian Formalization of the Update Step . . . 30

3 Sequential Model Based Optimization **33**

4 Surrogate Models for SMBO **35**
- 4.1 Introduction . . . 35
- 4.2 A Survey of Existing Surrogate Models . . . 35
  - 4.2.1 Gaussian Processes . . . 35
  - 4.2.2 Tree Parzen Estimators . . . 37
  - 4.2.3 Bayesian Neural Networks . . . 38
  - 4.2.4 Random Forest, Gradient Boosted Machines . . . 39
- 4.3 Other Techniques for Optimizing Hyperparameters . . . 40
  - 4.3.1 Grid Search . . . 40
  - 4.3.2 Random Search . . . 40
  - 4.3.3 Genetic Algorithms . . . 41

5 BASS and their Origins **43**
5.1 Decision Trees . . . 44
5.2 MARS Models . . . 45
5.3 BMARS Models . . . 49
5.4 BASS Models . . . 50
5.5 BASS Models as Surrogates in Bayesian Optimization . . . 52
5.5.1 Starting Point for the Algorithm . . . 53
5.5.2 Discretization of the Function Domain . . . 53
5.5.3 Uncertainty Estimation . . . 54

6 Experimental Validation **57**
6.1 Introduction . . . 57
6.2 Experimental Design . . . 58
6.2.1 Objective and Scope . . . 58
6.2.2 Hyperparameter Optimization Methods . . . 58
6.2.3 Experimental Procedure . . . 59
6.2.4 Evaluation . . . 60
6.2.5 Statistical Analysis . . . 60
6.2.6 Initial Testing Conditions . . . 60
6.2.7 Acquisition and Candidate Generation . . . 61
6.2.8 TPE Sensitivity to Its Gamma Hyperparameter . . . 62
6.3 Results . . . 63
6.3.1 Branin-Hoo Function . . . 63
6.3.2 Rastrigin Function . . . 64
6.3.3 Synthetic Non-Smooth Surface . . . 65
6.4 Categorical and Mixed-Variable Benchmarks . . . 66
6.4.1 Func-2C and Func-3C (mixed) . . . 68
6.4.2 Cat-Ackley (purely categorical) . . . 69
6.4.3 TPE Hyperparameter Sensitivity . . . 72
6.5 Result Interpretation and Limitations . . . 73

7 Conclusions and Future Work **79**
7.1 Conclusions . . . 79
7.2 Future Work . . . 82

Introduction

Many of the most important optimization problems in machine learning and the sciences share an uncomfortable feature: the function we want to optimize is a black box that is expensive to evaluate. Tuning the hyperparameters of a learning algorithm, calibrating a simulator, or configuring a physical experiment all amount to searching for the input that minimizes some objective whose internal structure we cannot see and whose every evaluation costs real time or money. Bayesian Optimization (BO) has become the standard tool for this setting because it spends evaluations frugally: it fits a cheap probabilistic *surrogate* to the data gathered so far and uses that surrogate, through an acquisition function, to decide where to evaluate next. The quality of a BO method is therefore largely the quality of its surrogate, and the default choice, by a wide margin, is the Gaussian process (GP).

The Gaussian process owes its dominance to genuine strengths, chief among them a closed-form predictive uncertainty that the acquisition function can use directly. But that machinery rests on an assumption that is easy to overlook: a GP is built from a kernel that measures the similarity of two inputs through a numeric distance between them, and so it presumes the search space is a continuous subset of $\mathbb{R}^d$. A great many real problems are not of this form. Hyperparameter spaces routinely mix continuous parameters with *categorical* ones, such as the choice of model family, kernel, or optimizer, whose values are unordered labels with no natural distance between them. Faced with such inputs a standard GP must resort to encodings that fabricate an ordering or distort the geometry its kernel relies on, or to specialized kernels and embeddings that are additional machinery rather than part of the model.

This thesis investigates an alternative surrogate that does not share this restriction: the Bayesian Adaptive Spline Surface (BASS). BASS is a fully Bayesian spline-regression model in the lineage of MARS and Bayesian MARS, and it handles categorical predictors *natively*, through basis functions defined on subsets of a factor's levels rather than on numeric distances. The central contribution of this work, as its title suggests, is the implementation

and study of BASS as a surrogate model within the Bayesian Optimization framework, with particular attention to the capability that distinguishes it from the GP baseline: optimizing over categorical and mixed search spaces. In the language of the next chapters, the point of this work is to relax the continuity-and-numeric-inputs restriction that the GP framework quietly imposes.

To study this honestly, we build a reproducible experimental pipeline in R that compares BASS-based BO (BASS-BO) against GP-based BO (GP-BO), Random Search, and the Tree-structured Parzen Estimator (TPE) on a common protocol, with both surrogates driven by the same parameter-free Expected Improvement acquisition and the same candidate generator, so that any measured difference is attributable to the surrogate alone. The framing is deliberately balanced rather than advocacy-driven, and the outcomes are reported whichever way they fall. On the smooth, low-dimensional continuous benchmarks, where the GP's assumptions are well matched, GP-BO is indeed the stronger method — decisively so, winning every paired seed against BASS-BO on Branin — while BASS-BO still clearly improves on Random Search. On purely categorical problems the structural capability is demonstrated: BASS-BO solves a solvable categorical benchmark on every seed and consistently outperforms both Random Search and TPE as the space grows past what the budget can cover, although a GP on a plain continuous relaxation, given the same categorical-aware search machinery, proves nearly as effective. On mixed continuous–categorical problems the result is negative: BASS-BO fails to beat Random Search there, a finding we report and analyze rather than obscure. The experimental chapter presents all of these results with paired per-seed statistics, and the conclusions draw the regime map they imply.

The remainder of the thesis is organized as follows. We first develop the background needed to make these ideas precise: stochastic processes and Gaussian processes, the Bayesian Optimization framework and its acquisition functions, and the sequential model-based optimization (SMBO) view that unifies them. We then survey the surrogate models commonly used in this framework and their trade-offs, before turning to BASS itself, its origins in MARS and Bayesian MARS, and the specifics of using it as a surrogate, including how it handles categorical inputs and how its posterior supplies the uncertainty an acquisition function needs. Finally, we present the experimental validation across both continuous and categorical benchmarks, and conclude with a discussion of where each method is preferable and what should come next.

There is not a single theorem, lemma, or proposition in the whole paper. Have my senses taken leave of me? What, no asymptotics or results concerning the rate at which MARS approaches the "True Model" as the sample size goes to infinity? For one of the few times in its history, the Annals of Statistics has published an article based only on the fact that this may be a useful methodology. All is ad hoc; there is no maximum likelihood, no minimax, no rates of convergence, no distributional or function theory. Is nothing sacred? What kind of statistical science is this? My thanks go to the editor and the others involved in this sacrilegious departure.

- Leo Breiman, discussing the MARS model.

Chapter 1

Stochastic Processes

Before starting, we would like to note that sections of this chapter where constructed using different references as primary guides. The section on Markov chains primarily uses the references [Žitković, 2010] and [Hohendorff and Rosenthal, 2005].

Stochastic processes play a very important role in basically every part of this work. First, the basis of traditional Bayesian Optimization is Gaussian processes, which are a specific class of stochastic process in which all the points or vectors have a conjoined multi-normal distribution. This is not the only place where stochastic processes play a role, as BASS is a Bayesian method and modern Bayesian methods generally rely on simulation strategies such as Markov Chain Monte Carlo (MCMC) to numerically approximate complex posterior distributions. We use a particular class of MCMC algorithms called reverse jump MCMC when dealing with our alternate surrogate function, but that is detailed much later when we talk about the Bayesian Adaptive Regression Spline method.

This chapter develops the theoretical foundations required to construct and reason about Gaussian processes, with the goal of arriving at Gaussian process regression as a principled framework for supervised learning under uncertainty. The exposition follows a deliberate progression: we begin with the theory of Markov chains, establish the ergodic properties that justify their use in sampling-based inference, and then build toward Gaussian processes as a continuous generalization of the finite–dimensional Gaussian distribution. Central to this development is the role of the kernel function, which encodes prior assumptions about the structure of the process and determines the qualitative behavior of its realizations. The chapter concludes with Gaussian process regression, which is the mechanism by which uncertainty is quantified and function approximations are constructed in the traditional approach to Bayesian optimization.

1.1 General Definitions

All in all, stochastic processes are generally important to the content of this work so we include some important results and definitions, starting with the definition of a stochastic process itself, taken from [Žitković, 2010].

Definition 1. *Let $\Theta \subseteq \mathbb{R}^+$. A stochastic process $\{X_\theta, \theta \in \Theta\}$ is a collection of random variables, indexed by a parameter θ, such that θ belongs to some index set Θ. When $\Theta = \mathbb{N}$ (or $\Theta = \mathbb{N}_0$), then it is called a discrete-time process, and if $\Theta = \mathbb{R}^+$, then it is called a continuous-time process.*

To distinguish between a discrete and continuous time process, we use the index n for discrete-time processes and t for continuous-time processes. In all applications of this work the indexes refer to time, as the function spaces $\mathcal{X}$ on which the functions are evaluated are time dependent. This definition is very broad, as for example, if we take the case where $\Theta = \{1\}$, then the stochastic process $\{X_\theta, \theta \in \Theta\}$ is really just X_1, a random variable. As with many other areas of mathematics though, the introduction of infinities in the values the indexes can take introduces a large amount of complexity, and things like vector distributions do no extend nicely to these new cases.

Another more formal definition of a stochastic process defines it as a function of the index and an element of the sample space, as follows:

Definition 2. *Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space[1] where Ω is a sample space, $\mathcal{F}$ is a σ-algebra on Ω and $\mathbb{P} : \mathcal{F} \mapsto [0, 1]$ a probability measure. Taking Θ an arbitrary index space, a stochastic process is a function*

$$X : \Omega \times \Theta \mapsto \mathbb{R}$$

such that for all $\theta \in \Theta$,

$$X_\theta : \omega \mapsto X(\omega, \theta) \equiv X_\theta : \Omega \mapsto \mathbb{R}$$

is a measurable function, which is equivalent to saying that X_θ is a random variable.

When ω is fixed, the map $\theta \mapsto X(\omega, \theta) : \Theta \to \mathbb{R}$ is called a **sample function**, or a **realization** of the stochastic process. Geometrically, each realization traces a trajectory which is a single path through the range of X. These types of trajectories are of the kind shown in Figure 1.1, which displays 100 simulated realizations of a Gaussian random walk, a more concrete example of these types of functions. A natural application of this framework

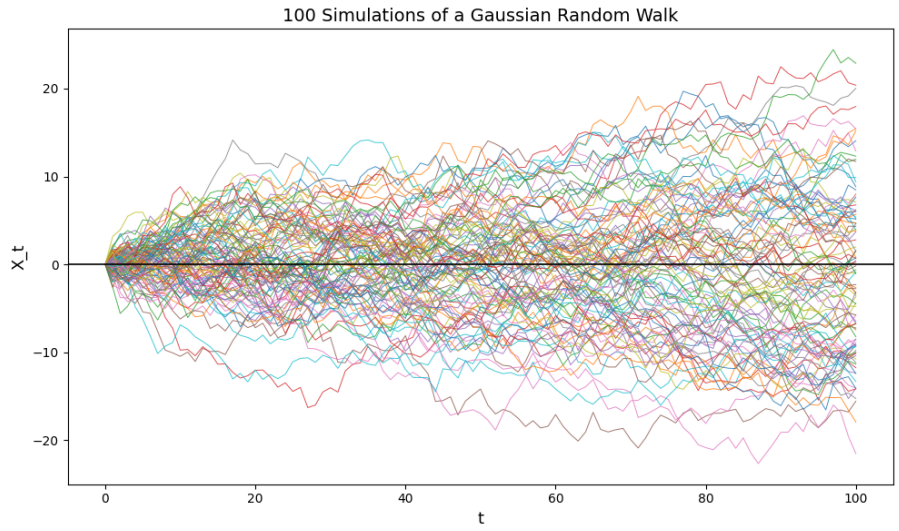

Figure 1.1: Random walks across a parameter t, where Θ is taken to be a time index set (with elements conventionally denoted t), and each realization represents one possible evolution of the process over time.

is **time series analysis**, where Θ is taken to be a time index set and each realization represents one possible evolution of the process over time.

As was mentioned earlier, the two main types of stochastic process that can be defined are ones where the realizations of the process occur in a discrete time, and the other is when they occur in a continuous fashion. It so happens that the two main structures relating to stochastic processes that we need to go into detail on are Markov chains, a discrete process, and Gaussian processes, which are continuous. We go in depth in the following sections into each of these types of stochastic processes, starting with Markov chains since Gaussian processes lead nicely onto the next chapter.

1.2 Markov Chains

Markov chains are ubiquitous in their applications, since they represent a relatively simple but very powerful concept: the mathematical structure on which we can define phase transitions of position changes that are governed by probabilistic rules. The fundamental property which will be explored later is that independently of the past, the only important factor in determining

[1]Billingsley's classic book [Billingsley, 2012] has all the theoretical foundation of probability spaces and provides a thorough run down of these concepts.

the future state of the process is the current state of it. We begin with a formal definition and build up the general theory of Markov Chains from there.

Definition 3. *Let* $\{X_n, n \in \mathbb{N}_0\}$ *be a stochastic process defined on a probability space* $(\Omega, \mathcal{F}, \mathbb{P})$ *such that*

$$X_n : \omega \mapsto X(\omega, n) \equiv X_n : \Omega \mapsto \mathcal{S},$$

where $\mathcal{S}$ *is a finite state space of cardinality* k*, such that* $\mathcal{S} = \{s_1, s_2, \ldots, s_k\}$*. We call* $\{X_n, n \in \mathbb{N}_0\}$ *a Markov chain if*

$$\mathbb{P}(X_n = s_n | X_{n-1} = s_{n-1}, X_{n-2} = s_{n-2}, \ldots, X_0 = s_0) = \mathbb{P}(X_n = s_n | X_{n-1} = s_{n-1})$$

for all $n \in \mathbb{N}_0$ *and all* $s_0, s_1, \ldots, s_k \in \mathcal{S}$ *whenever the two conditional probabilities are well defined, this is, when* $\mathbb{P}(X_{n-1} = s_{n-1}, \ldots, X_0 = s_0) > 0$*.*

Intuitively, this means that the next state in the chain (the sequence $\{X_n, n \in \mathbb{N}_0\}$) is only dependent its current state, and that the history of how the chain got to the current state is irrelevant to the next step. This behaviour is known as the Markov property, and one very valuable insight that emerges from this behaviour is being able to completely determine all properties of the chain given only the initial state of it and the knowledge of how the chain behaves at all of its possible states.

The initial state or initial probability is self-explanatory, it tells us the expected behaviour of the first element in the chain, X_0. Formally, we call $\mu^{(0)} = \{\mu_s^{(0)} : s \in \mathcal{S}\}$ with $\mu_s^{(0)} = \mathbb{P}(X_0 = s)$ the initial probability distribution of the process.

The other important component to fully describe a Markov chain is the transition probabilities from one state s_i to another s_j for all $s_i, s_j \in \mathcal{S}$. We identify this probability as

$$p_{i,j} = \mathbb{P}(X_n = s_j | X_{n-1} = s_i).$$

If we take all combinations of the states the chain can be in and map them to all other states, we get what is known as the transition matrix P, where each entry in the matrix denotes the probability of moving from one specific state to another. This matrix, because it is fundamentally a matrix made up of probabilities, has two interesting properties for the purposes of this work:

- $p_{i,j} \geq 0$ for all $i, j \in \{1, 2, \ldots, n\}$,
- $\sum_{i=1}^{n} p_{i,j} = 1$ for all $j \in \{1, 2, \ldots, n\}$.

We note that these two properties are the definition of a stochastic matrix, and therefore, by construction, Markov chain transition matrices are stochastic matrices. We have only described that we can fully determine the Markov chain from its initial state and its transition probabilities, but we have not shown this concept concretely. The following theorem ties these loose ends together and formally describes this property.

Theorem 1. *Let* $\{X_n, n \in \mathbb{N}_0\} : \Omega \mapsto \mathcal{S}$ *be a stochastic process defined on the probability space* $(\Omega, \mathcal{F}, \mathbb{P})$ *with* $\mathcal{S}$ *a finite state space. Then,* $\{X_n, n \in \mathbb{N}_0\}$ *is a Markov chain if and only if there exists a stochastic matrix* P *such that*

$$\mathbb{P}(X_n = s_n | X_{n-1} = s_{n-1}, \ldots, X_0 = s_0) = p_{n-1,n}$$

for all $n \in \mathbb{N}_0$ *and all* $s_0, s_1, \ldots, s_n \in \mathcal{S}$ *such that* $\mathbb{P}(X_{n-1} = s_{n-1}, \ldots, X_0 = s_0) > 0$.

This theorem is of great importance in the area of study of Markov chains, and we refer the interested reader to [Chung, 1960, §1.2] for a detailed proof. this theorem establishes a bridge between the theory of Markov processes in general and the properties of linear algebra and matrix theory. One such interesting property is the use of eigenvectors and eigenvalues of the transition matrix used to define a stationary distribution, which is what we are ultimately building towards this section.

We have seen that the transition matrix P governs the behaviour and the changes that happen from one state of the Markov chain to the next. A natural follow up question arises from trying to understand the behaviour of the chain an arbitrary $k > 1$ steps in the future instead of a single step, such as if we wanted to calculate $\mathbb{P}(X_{i+k} = s_{i+k} | X_i = s_i)$. We first note that this multi-step transition probability can be simplified, assuming that the Markov chain is homogenous [2], to the following:

$$\mathbb{P}(X_{i+k} = s_{i+k} | X_i = s_i) = \mathbb{P}(X_k = s_k | X_0 = s_0)$$

which we denote as $p_{i,k}^{(k)}$. Because we can define the individual probabilities of transitioning from one state to another after k steps, it stands to reason that there would be a transition matrix to describe this behaviour. The theorem that establishes the way these relationships work is the Chapman-Kolmogorov theorem and equation, which we present now. The formulation of this theorem presented in this work is taken from [Žitković, 2010, §8.3].

[2] A Markov chain is homogenous if the probability of transitioning from one state to another remains constant as we increase n.

Theorem 2 (Chapman-Kolmogorov Theorem)**.** *Let $n, m \in \mathbb{N}_0$ and $s_i, s_j \in \mathcal{S}$. Then, $p_{i,j}^{(n+m)}$, which is the probability that starting from state i at step n we end up in state j after m steps is given by:*

$$p_{i,j}^{(n+m)} = \sum_{k \in \mathcal{S}} p_{i,k}^{(n)} \, p_{k,j}^{(m)}.$$

Proof. We will prove the case when the Markov process is homogenous. We begin by noting that $p_{i,j}^{(n+m)} = \mathbb{P}(X_{n+m} = s_j | X_n = s_i) = \mathbb{P}(X_m = s_j | X_0 = s_i) = p_{i,j}^{(m)}$. Applying the law of total probability and using the Markov property,

$$\begin{aligned} p_{i,j}^{(n+m)} &= \\ &= \mathbb{P}(X_{n+m} = s_j | X_n = s_i) \\ &= \mathbb{P}(X_m = s_j | X_0 = s_i) \\ &= \sum_{k \in \mathcal{S}} \frac{\mathbb{P}(X_m = s_j, X_n = s_k, X_0 = s_i)}{\mathbb{P}(X_0 = s_i)} \\ &= \sum_{k \in \mathcal{S}} \mathbb{P}(X_m = s_j | X_n = s_k) \mathbb{P}(X_n = s_k | X_0 = s_i) \\ &= \sum_{k \in \mathcal{S}} p_{i,k}^{(n)} \, p_{k,j}^{(m-n)}. \end{aligned}$$

□

We now introduce the following lemma that takes advantage of this property to provide an alternative way of computing and understanding the transition matrix P.

Lemma 1. *Let P^k be the k-th order matrix power of the transition matrix P for a homogeneous Markov chain $\{X_n, n \in \mathbb{N}_0\}$. Then,*

$$p_{i,j}^{(k)} = (P^k)_{i,j} \text{ for } i, j \in \mathcal{S}.$$

There is a way to prove this lemma by induction directly, but we opt instead to use the Chapman-Kolmogorov Equation n-1 times to achieve the final result, as follows.

This lemma becomes useful in proving the following theorem.

Theorem 3. *Let $n, m \in \mathbb{N}_0$ and $i, j \in \mathcal{S}$. Then, $p_{i,j}^{(n+m)}$, which is the probability that starting from state i in step n we end up in state j after m steps is given by:*

$$p_{i,j}^{(n+m)} = \sum_{k \in \mathcal{S}} p_{i,k}^{(m)} \cdot p_{k,j}^{(n)}.$$

From here, we now present two definitions relating to the properties of matrices which will become important when further exploring the types of Markov chains we are interested in.

Definition 4. *A matrix $A \in \mathbb{R}^{n\times m}$ is called non-negative if $a_{i,j} \geq 0$ for all $1 \leq i \leq n$ and $1 \leq j \leq m$.*

Before continuing, we note that with stochastic matrices being just glorified assortments of probabilities that are neatly ordered, they are always non-negative.

Definition 5. *Let A be a non-negative matrix. If there exists $N_0 \geq 1$ such that all elements of A^{n_0} are positive, then we call A a quasi-positive matrix.*

These two definitions are necessary building blocks to get to the property we are truly interested in, ergodicity.

Definition 6. *A Markov chain $\{X_n : n \in \mathbb{N}_0\}$ is called ergodic if the limit $\pi_j := \lim_{n\to\infty} p_{i,j}^{(n)}$ exists for all $j \in \{1, 2, \ldots, k\}$, is positive and does not depend on i, and $\pi = (\pi_1, \pi_2, \ldots, \pi_k)$ is a probability distribution on $\mathcal{S}$.*

Ergodicity is a powerful property that does not arise intrinsically within the theory of stochastic processes, but is instead inherited from measure theory. Informally, a system is ergodic if the probability laws governing it remain invariant or unchanged by transitions between states or, in the case of time-indexed processes, by the passage of time. As a concrete illustration, consider tossing a fair coin 100 times. The distribution of outcomes observed by a single person performing all 100 tosses is, on average, identical to that obtained from 100 independent people each tossing the coin once; the underlying probability law is the same in both cases, and this system is ergodic. By contrast, suppose the coin is replaced by a biased one after 50 tosses. The probability law now changes mid-process and it is therefore not invariant across time or states which means that the system is therefore non-ergodic.

A thorough treatment of ergodicity lies beyond the scope of this work; the interested reader is directed to Sections 24 and 36 of [Billingsley, 2012] for a rigorous development through the lenses of measure theory and stochastic processes, respectively. At its core, ergodicity formalizes the equivalence between time averages and ensemble averages: rather than observing many independent realizations of a system simultaneously, one can instead observe a single realization over a long period of time and recover the same statistical quantities. This equivalence is precisely what makes ergodic Markov chains computationally tractable. The result that makes this precise, and upon

which the theoretical justification for Markov Chain Monte Carlo (MCMC) methods rests, is the Ergodic Theorem, stated below.

Theorem 4. *Let $\{X_n : n \in \mathbb{N}_0\}$ be an ergodic Markov chain such that*

$$\pi_j = \lim_{n\to\infty} p_{i,j}^{(n)}, \quad \textit{for all } j \in \mathcal{S}.$$

Then the vector π is the unique solution of

$$\pi^T = \pi^T P,$$

and π is a probability distribution on $\mathcal{S}$.

Proof. Let j be an arbitrary element of $\mathcal{S}$. By the Chapman–Kolmogorov equations (Theorem 3),

$$p_{i,j}^{(n+1)} = \sum_{k\in\mathcal{S}} p_{i,k}^{(n)}\, p_{k,j}.$$

Taking the limit as $n \to \infty$ and applying $\pi_k = \lim_{n\to\infty} p_{i,k}^{(n)}$ for each $k \in \mathcal{S}$ gives

$$\pi_j = \sum_{k\in\mathcal{S}} \pi_k\, p_{k,j}.$$

Since $j \in \mathcal{S}$ was arbitrary, this holds for all j, which is precisely the component-wise form of $\pi^T = \pi^T P$.

To verify that π is a probability distribution: since $p_{i,j}^{(n)} \geq 0$ for all n, we have $\pi_j \geq 0$ for all $j \in \mathcal{S}$. Furthermore, since $\sum_{j\in\mathcal{S}} p_{i,j}^{(n)} = 1$ for all n, taking the limit yields $\sum_{j\in\mathcal{S}} \pi_j = 1$. Hence π is a valid probability distribution on $\mathcal{S}$.

Uniqueness follows from the ergodicity assumption: an ergodic (irreducible and aperiodic) chain admits exactly one stationary distribution, so π is the unique solution of $\pi^T = \pi^T P$.

□

If we construct a Markov chain in a way that it is ergodic an control the distribution π so that it is something that we want, we can guarantee that after a sufficiently large amount of iterations n we will be sampling from the stationary distribution of the process, which is invariant through time because of the property presented in the above theorem. This is the crux of understanding the Markov Chain Monte Carlo technique which is the central object of study of this section. Further diving into the construction of the Metropolis–Hastings algorithm and the fundamental building blocks

needed to construct these chains that eventually lead to stationary distributions we care about would make this work exceedingly long, but [Rubinstein and Kroese, 2016] and [Robert and Casella, 1999] are good resources for a detailed rundown of these topics. We will return much later in this work when discussing the BASS method to MCMC and why it is so essential to this process. We will not look at pure MCMC though but an interesting modification to the base algorithm known as the Reversible Jump Markov Chain Monte Carlo (RJMCMC) algorithm.

For now though we move on to the other interesting sub-case of stochastic processes relevant to this work, the continuous-time Gaussian Process.

1.3 Gaussian Processes

A Gaussian process can be thought of as a generalization of the Gaussian (normal) distribution when viewed from the lens of stochastic processes. Stochastic processes, as we saw before, are indexed by a parameter, which itself belongs to an index set. The Gaussian process extends the definition of the Gaussian distribution from being valued on scalars or vectors to a more general index set of the form $T \subseteq \mathbb{R}^+$. Note that this set can contain an infinite number of points, which is the main difference between the Gaussian distribution and the Gaussian process. This extension is not trivial though, and to specify the structure of these particular types of stochastic processes, we aid ourselves with finite dimensional distributions. We now provide a formal definition of these types of distributions.

Definition 7. *Let $\{t_i\}_{i\leq k}$ be a sequence of points*[3] *such that $\{t_i\}_{i\leq k} \subseteq T$. The k-dimensional random vector $(X_{t_1}, X_{t_2}, \ldots, X_{t_k})$ has a distribution $\mu_{t_1,t_2,\ldots,t_k}$ over $\mathbb{R}^k$. These measures $\mu_{t_1,t_2,\ldots,t_k}$ are the finite dimensional distributions of the process.*

One logical follow up is to ask if the inverse of this definition is also true. This would be, starting from a consistent[4] joint distribution of the random variables $(X_{t_1}, X_{t_2}, \ldots, X_{t_k})$, with $k \in \mathbb{N}$, then would it be possible to define a probability measure $\mu_{t_1,t_2,\ldots,t_k}$ on a higher cardinality set, in this case $\mathbb{R}^k$ that matches these distributions exactly? The positive answer to this question is the Kolmogorov Extension Theorem, an incredibly deep and powerful result that guarantees the existence of stochastic processes by only defining

[3] Sometimes referred to as time points, considering this index not as an abstract set but a representation of the progress of time as indexed by a subset of the positive reals.

[4] Consistency is a fuzzy term with a lot of meanings, but its meaning in this context will become clear with the theorems that follow.

a finite set of distributions that share some consistency requirements. The formulation of this theorem presented in this work is taken from [Øksendal, 2003, Theorem 2.1.5].

Theorem 5 (Kolmogorov Existence Theorem)**.** *Let T denote some interval, and let $n \in \mathbb{N}$. For each $k \in \mathbb{N}$ and finite sequence of distinct times $t_1, \ldots, t_k \in T$, let $\nu_{t_1 \ldots t_k}$ be a probability measure on $(\mathbb{R}^n)^k$. Suppose that these measures satisfy two consistency conditions:*

1. *For all permutations π of $\{1, \ldots, k\}$ and measurable sets $F_i \subseteq \mathbb{R}^n$,*

$$\nu_{t_{\pi(1)} \ldots t_{\pi(k)}} \left(F_{\pi(1)} \times \ldots \times F_{\pi(k)} \right) = \nu_{t_1 \ldots t_k} \left(F_1 \times \ldots \times F_k \right);$$

2. *For all measurable sets $F_i \subseteq \mathbb{R}^n$, $m \in \mathbb{N}$,*

$$\nu_{t_1 \ldots t_k}(F_1 \times \ldots \times F_k) = \nu_{t_1 \ldots t_k, t_{k+1}, \ldots, t_{k+m}} \left(F_1 \times \ldots \times F_k \times \underbrace{\mathbb{R}^n \times \ldots \times \mathbb{R}^n}_{m} \right).$$

Then there exists a probability space $(\Omega, \mathcal{F}, \mathbb{P})$ and a stochastic process $X : T \times \Omega \to \mathbb{R}^n$ such that

$$\nu_{t_1 \ldots t_k}(F_1 \times \ldots \times F_k) = \mathbb{P}(X_{t_1} \in F_1, \ldots, X_{t_k} \in F_k)$$

for all $t_i \in T$, $k \in \mathbb{N}$, and measurable sets $F_i \subseteq \mathbb{R}^n$, i.e., X has $\nu_{t_1 \ldots t_k}$ as its finite-dimensional distributions relative to times $t_1 \ldots t_k$.

The Kolmogorov Extension Theorem addresses the question of whether we can construct a stochastic process given only a finite set of joint distributions. These joint distributions need to satisfy a consistency condition, which means that any finite subset of random variables should have joint distributions that are compatible with each other.

The theorem states that if such consistent joint distributions exist, then it is possible to define a stochastic process on a higher-dimensional space, often denoted as $\mathbb{R}^k$, such that the original given distributions match the marginal distributions of this process. In essence, the Kolmogorov Extension Theorem provides a way to create a stochastic process that captures the dependencies and correlations between random variables at different time points, even when we only have limited information about their joint distributions.

In the case of Gaussian processes, this means that if we have a finite set of random variables following consistent joint Gaussian distributions, we can construct a Gaussian process on a continuous domain (typically a subset of $\mathbb{R}$) such that any finite subset of the process is jointly Gaussian with the desired covariance structure. With this auxiliary result out of the way, we can now proceed to define the Gaussian process formally.

Definition 8. *Let $\mathcal{X}$ be a nonempty index set, typically taken to be a subset of $\mathbb{R}^d$ for some $d \geq 1$. Let $\{X_t : t \in \mathcal{X}\}$ be a stochastic process. We call $\{X_t : t \in \mathcal{X}\}$ a Gaussian process if for every finite collection $\{t_i\}_{i \leq k} \subset \mathcal{X}$ and every finite k, the random vector $(X_{t_1}, X_{t_2}, \ldots, X_{t_k})$ follows a multivariate Gaussian distribution when $k > 1$ and a univariate Gaussian distribution when $k = 1$.*

An important extension of this is that we can interpret the Gaussian process as a random function f evaluated at input points $x \in \mathcal{X}$. That is, for each $x \in \mathcal{X}$, there is a random variable $f(x)$ representing the possible output of the function f at that location, or equivalently, a realization of a Gaussian distribution at x. This can also be understood as drawing a sample from the distribution of $f(x)$. As with any Gaussian distribution, $f(x)$ is completely characterized by its mean and variance. Unlike the finite-dimensional case, however, the index set $\mathcal{X}$ may contain infinitely many points, and so the natural approach is to characterize the process through a mean function and a covariance function defined over $\mathcal{X}$. We denote the mean function of the process as:

$$m(x) = \mathbb{E}[f(x)]$$

and the covariance function as:

$$k(x, x') = \mathbb{E}[(f(x) - m(x))(f(x') - m(x'))].$$

The main reason why the covariance function is denoted by k is that this name alludes to the fact that k is a kernel of the terms in the input space. We now present a formal definition of this type of function, followed by some intuition on its interpretation. With these definitions though, we write the distribution function for f as

$$f(x) \sim \mathcal{GP}(m(x), k(x, x')).$$

Definition 9. *Let $\mathcal{X} \subseteq \mathbb{R}^d$ be a nonempty input space, and let V be a vector space equipped with an inner product $\langle \cdot, \cdot \rangle_V$. Let $\varphi : \mathcal{X} \to V$ be a mapping, called a feature map. We call the function $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ a kernel function if*

$$k(x, x') = \langle \varphi(x), \varphi(x') \rangle_V, \quad \text{for all } x, x' \in \mathcal{X}.$$

What this kernel function allows us to do is essentially borrow an inner product from the feature space V and apply it to transformed elements of the input space $\mathcal{X}$. This allows us to compute inner products in a potentially high-dimensional or implicit feature space without requiring explicit knowledge of its structure. Tying the kernel back to Gaussian processes, we note

that while the mean function controls only the average value across realizations of f, the covariance function (that is, the kernel) governs the underlying behavior and structure of those realizations. Different choices of kernel encode different prior assumptions about the process, such as smoothness, periodicity, or stationarity. The rational quadratic kernel is a noteworthy case: it can be interpreted as a scale mixture of RBF kernels with different length scales, and interpolates between smooth and rough sample path behavior depending on the parameter α. Figure 1.2 illustrates how the choice of kernel affects the character of sample paths drawn from a Gaussian process.

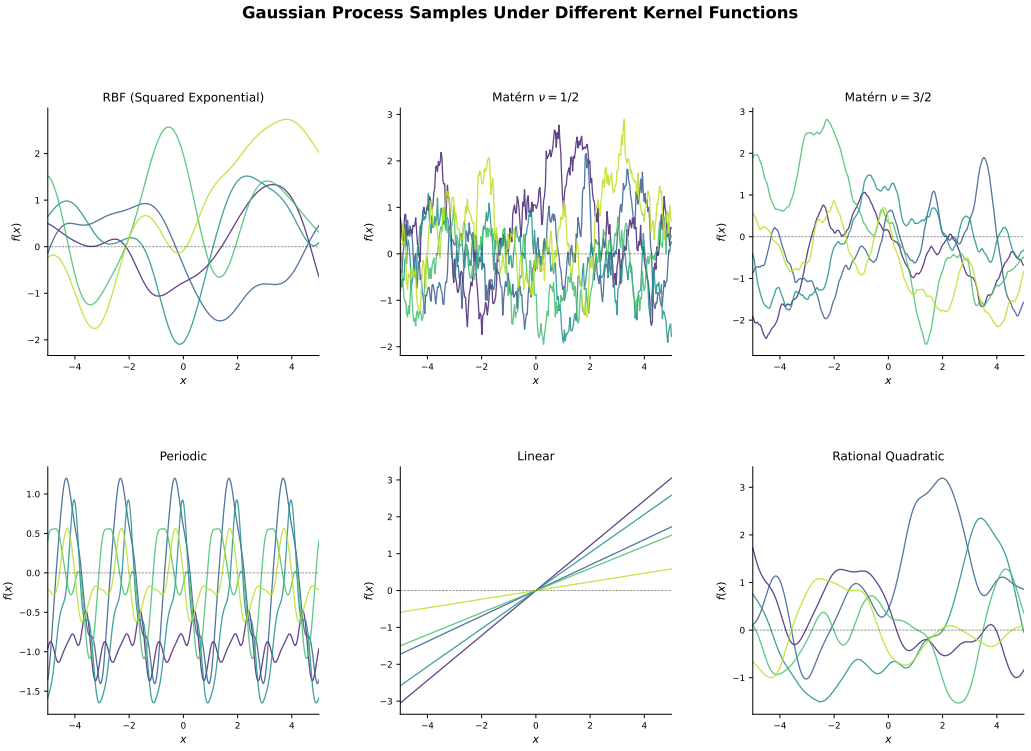

Figure 1.2: Sample paths from a zero-mean Gaussian process under six kernel functions: the RBF (squared exponential), Matérn $\nu = 1/2$, Matérn $\nu = 3/2$, periodic, linear, and rational quadratic kernels. Each colored trajectory represents an independent sample from the process.

Basically all major properties such as smoothness, differentiability, and range (to name only a few) are controlled by the choice of the kernel function. While the kernel function can be basically anything that fulfils the above definition 9, there are some particularly useful or popular kernel functions when dealing with Gaussian processes. Chief among them is the squared exponential kernel, also known as the radial basis function. This kernel is

defined as

$$k(x, x') = \sigma_f^2 \exp\left(\frac{-\ell(x - x')^2}{2}\right),$$

where σ_f^2 and ℓ are hyperparameters. While we do not go into too much detail as to why this is a good choice for a kernel, we refer the interested reader to [Garnett, 2023]. Note that this kernel, like the others in common use, depends on the inputs only through the numeric difference $(x - x')$, and so presumes that the index set is a subset of $\mathbb{R}^d$ as in the definitions given above. This is a comfortable assumption for continuous problems but a restrictive one for categorical inputs, a limitation we return to when we compare surrogate models and when we motivate the use of BASS.

The hyperparameters defined previously each control the general behaviour of the process when fitting data points. A more detailed run down of these points is given in the next section, which talks about Gaussian process regression. This is the method by which uncertainty is measured and function approximations are conducted in the traditional approach to Bayesian optimization.

1.4 Gaussian Process Regression

Gaussian process regression is the primary application of the theory developed in this chapter. It is a nonparametric approach to supervised learning in which, rather than fitting a fixed parametric model to data, one places a prior distribution directly over functions and updates it in light of observed data. Concretely, given a set of input-output pairs $\{(x_i, y_i)\}_{i=1}^n$, Gaussian process regression produces a posterior distribution over functions consistent with those observations, providing not only a point prediction at new inputs but a calibrated measure of uncertainty. It is this uncertainty quantification that makes Gaussian process regression particularly well-suited to Bayesian optimization, where the balance between exploration and exploitation depends critically on knowing where the model is uncertain.

The method generates from data a mean function and then uses that mean function to extrapolate or predict on points where there is no data observed. As we described in the last chapter, we can fully specify or determine a particular Gaussian process by specifying its mean $m(x) = \mathbb{E}[f(x)]$ and its kernel function $k(x, x')$. While in the last chapter we started with the Gaussian process and from that generate the values on the rest of the input space, in this chapter we are going to be doing somewhat of the inverse of that. We will be starting from a set of data $\{x_i\}_{i \leq n} = \mathcal{D} \subset \mathcal{X}$, and computationally estimating the parameters of the kernel and mean to achieve the

best fit Gaussian process to the points and minimize the error.

One of the advantages of Gaussian process regression over other forms of supervised learning algorithms (such as linear regression) is the fact that Gaussian processes are a non-parametric model. Without delving too much into the specifics, this means that the number of parameters controlling the model is not fixed at a certain quantity, but rather is adaptive. The amount of parameters can grow and shrink depending on the data that is being fit.

This however does not mean that we do not have control over the behavior of the function we decide to adjust to the points. In the following figure, we can see how changing the hyperparameter ℓ in the squared exponential covariance function changes the relative smoothness of the fit function.

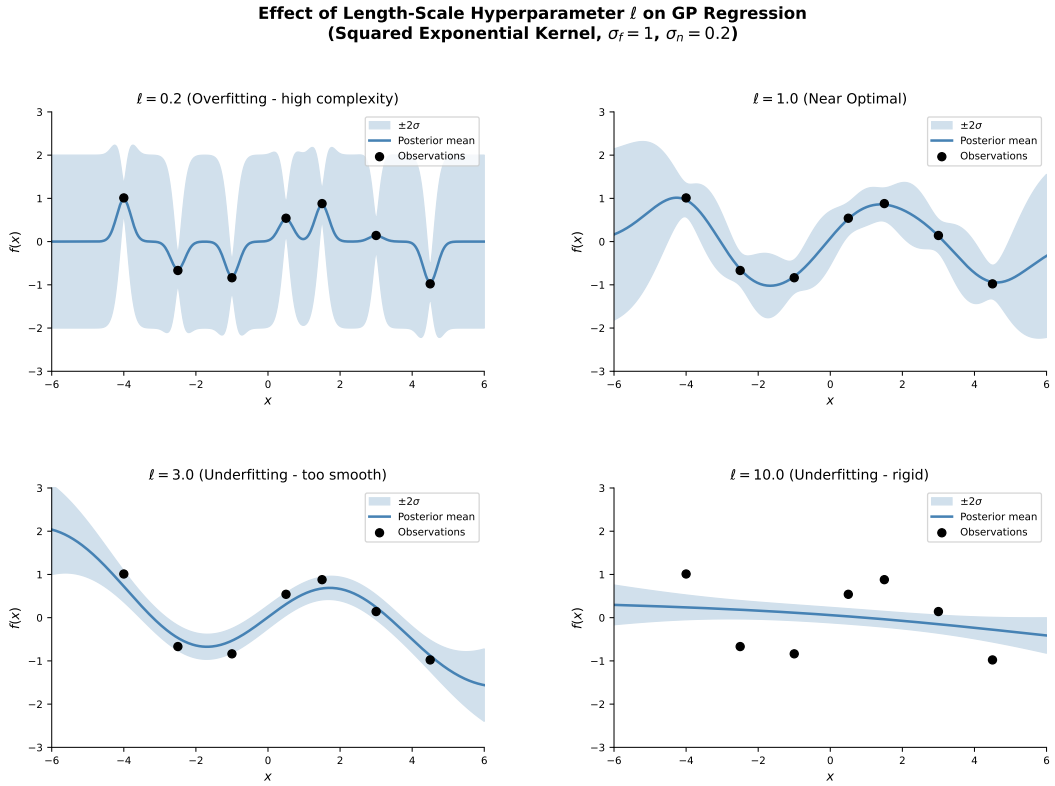

Figure 1.3: How different values of ℓ change the behaviour of the Gaussian process.

We also note how radically the confidence intervals change from one adjustment to the next based on moving this hyperparameter around. Because we can define an infinite amount of Gaussian processes based on our choice of hyperparameters (and further even by changing the kernel function), we

require a principled criterion for selecting among them. The standard approach is to select the hyperparameters that best explain the observed data in a probabilistic sense, formalized as follows.

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ be the observed data, and suppose observations are generated according to

$$y_i = f(x_i) + \varepsilon_i, \quad \varepsilon_i \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma_n^2),$$

where $f \sim \mathcal{GP}(m, k_{\boldsymbol{\theta}})$ and $\boldsymbol{\theta}$ denotes the vector of hyperparameters. Let $\mathbf{y} = (y_1, \ldots, y_n)^T$ and let $K_{\boldsymbol{\theta}} \in \mathbb{R}^{n \times n}$ denote the kernel matrix with entries $(K_{\boldsymbol{\theta}})_{ij} = k_{\boldsymbol{\theta}}(x_i, x_j)$. Under the Gaussian process prior and the additive noise model, the marginal distribution of the observations is

$$\mathbf{y} \mid X, \boldsymbol{\theta} \sim \mathcal{N}\big(\mathbf{0},\, K_{\boldsymbol{\theta}} + \sigma_n^2 I\big)\,,$$

where we assume a zero mean function without loss of generality. Denoting $C_{\boldsymbol{\theta}} = K_{\boldsymbol{\theta}} + \sigma_n^2 I$, the log marginal likelihood is

$$\log p(\mathbf{y} \mid X, \boldsymbol{\theta}) = -\frac{1}{2}\mathbf{y}^T C_{\boldsymbol{\theta}}^{-1}\mathbf{y} - \frac{1}{2}\log\det C_{\boldsymbol{\theta}} - \frac{n}{2}\log 2\pi.$$

The three terms admit a natural interpretation: the first is a data-fit term that penalizes poor prediction of the observations, the second is a complexity penalty that discourages unnecessarily flexible models, and the third is a normalizing constant. Maximizing the log marginal likelihood over $\boldsymbol{\theta}$ is equivalent to the minimization problem

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \left[\frac{1}{2}\mathbf{y}^T C_{\boldsymbol{\theta}}^{-1}\mathbf{y} + \frac{1}{2}\log\det C_{\boldsymbol{\theta}}\right], \tag{1.1}$$

where the constant term has been dropped as it does not depend on $\boldsymbol{\theta}$. This objective automatically balances data fit against model complexity, a property sometimes referred to as the *Occam's razor* effect of the marginal likelihood (more on this in §5.2 of [Rasmussen and Williams, 2006]). In the case of the squared exponential kernel, $\boldsymbol{\theta} = (\ell, \sigma_f^2, \sigma_n^2)$, and (1.1) is typically minimized via gradient-based methods, as the gradient of the objective with respect to each hyperparameter can be computed analytically.

The minimization problem stated in (1.1) thus provides the formal criterion for hyperparameter selection: we seek the $\hat{\boldsymbol{\theta}}$ that makes the observed data most probable under the model. This problem translates to finding the Gaussian process that best explains the observations in the sense that it minimizes residual uncertainty while penalizing unnecessary complexity.

The solution involves selecting the optimal[5] set of hyperparameters given the observed data, and using the resulting mean function to predict values at unobserved inputs $\boldsymbol{x}_*$.

To arrive at this more concretely, we will begin by reviewing regression in general terms and then establish the formal link to Gaussian process regression, showing how the framework developed above gives rise to a fully specified supervised learning method.

1.4.1 Preamble to Gaussian Process Regression: A Rehash on Regression Analysis

The central task of this section is regression, which is the process of estimating or generating understanding of a theoretical function $f : \mathcal{X} \to \mathbb{R}$ that describes the relationship between a response variable y and a group or single input variable described by the vector $\boldsymbol{x}$. While we do not know the exact behavior of f the function that describes this relationship[6], what we do have is a set of observations $\mathcal{D} \subset \mathcal{X}$, where $\mathcal{D} = \{(x_i, y_i), i = 1, 2, \ldots, n\}$ and $x_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$. These observations constitute point-wise realizations of the function f. From that, for a set of new unknown points $\boldsymbol{x}_*$ we want to predict or infer the quantity $f(x_{i*}) = y_*$, which would be point-wise estimates of these unseen or unknown realizations of f.

One example of another method of solving this problem is the well known and well loved linear regression, that assumes that the function f is of the general form

$$y = f(\boldsymbol{x}) = \boldsymbol{x}^T\boldsymbol{\beta} + \epsilon$$

where $\epsilon \sim \mathcal{N}(0, \sigma^2)$. The error variance σ^2 and the coefficients $\boldsymbol{\beta}$ are generated by a process of minimizing the error of the outputs generated.

One of the disadvantages of classical regression techniques such as linear regression is that they only give one function as the output, but there may well be an infinite number of functions that fit a certain set of points. As an example, consider the following figure.

Notice how for the training dataset $\mathcal{D}$, all of these functions have error zero! The only recourse we have is to restrict these functions to generate one function (or at least a finite family of functions), and not an infinity of

[5] Optimal is a dangerous word in this context. Depending on the problem and the structure of the marginal likelihood surface, the minimizer may not be unique, and in practice one often recovers a local rather than global minimum. Here we simply refer to the hyperparameters that solve (1.1), or a member of the family of solutions if multiple exist.

[6] We will discuss this in later chapters, but this is actually one of the greatest advantages of Bayesian Optimization, in that in can work on so called *black-box* functions.

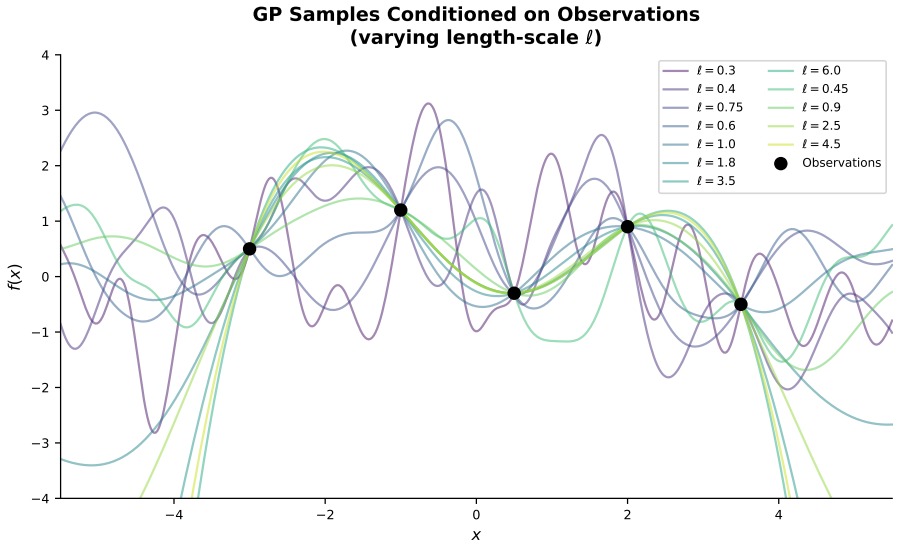

Figure 1.4: A sample of functions that could fit an arbitrary set of observations.

them. As with many other areas of mathematics, there is a trade off that we have to choose to perform between how general a solution to a problem is and how many solutions to that problem we wish to get.

The key insight to understanding Gaussian process regression from a statistician's point of view is changing the way we think about regression. Instead of going the traditional way and providing restrictions to generate particular approximations, why don't we consider the space of possible functions that can fit these points as coming from a probability model? What if the observed functions in the past figure are realizations of a more general class of random variable that is coherent in the function space, rather than the discrete variable or vector space? From this groundwork is where Gaussian processes come into the picture. The only added restriction that we impose on the functions is that as a family, they come from a generalization to infinite dimensions of the multivariate normal distribution, and with that, we generate the entire conceptual framework from which we derive and generate the Gaussian process regresion method. Having now set the expectation of what we wish to accomplish, we can now construct the theory of this different class of regression analysis.

1.4.2 Theory of Gaussian Process Regression (from the function space view)

This section of the text is heavily based on the work and structure of [Rasmussen and Williams, 2006] and to a much lesser extent [Wang, 2020]. The book Gaussian Processes for Machine Learning is the go-to reference and source for modern applications of Gaussian processes in general and it would be a disservice to the reader to not follow that structure and argumentation.

We can split up the set $\mathcal{D}$ into training and test, defining $\mathcal{D}_{train} = \{x_i, y_i\}_{i=1}^{n}$, and $\mathcal{D}_{test} = \{x_i^*, y_i^*\}_{i=1}^{n^*}$, with $x_i, x_i^* \in \mathbb{R}^d$ and $y_i, y_i^* \in \mathbb{R}$. As described before, what we want to achieve is basically generating out of sample predictions for the test set D_{test}. Since the function $f = m$ defined is the mean of the process we defined before, we can simply evaluate the out of sample points x_i^* on f and we have an estimate for y_i^*. For simplicity and by convention, we denote the set of of test outputs $\boldsymbol{f}_*$. Because of the structure we have defined, formally, the regression function f is really a normal distribution when conditioned by the training dataset $\mathcal{D}_{train}$, which would be

$$\mathbb{P}(\boldsymbol{f}|\boldsymbol{X}) = \mathcal{N}(\boldsymbol{f}|\boldsymbol{\mu}, k)$$

where $\boldsymbol{X} = [x_1, x_2, \ldots, x_n]$, $\boldsymbol{f} = [f(x_1), f(x_2), \ldots, f(x_n)]$, $\boldsymbol{\mu} = [m(x_1), m(x_2), \ldots, m(x_n)]$ and k the kernel or covariance function of the process. By convention, it is customary to set $m(\boldsymbol{X}) = 0$, but there is a reasoning behind this. When dealing with the type of data generally modelled by these types of functions, it is common (encouraged even) to normalize the data before applying these methods to it. Another important piece of shorthand notation we will use is $K = k(\boldsymbol{X}, \boldsymbol{X})$, $K_* = k(\boldsymbol{X}, X_*)$, and $K_{**} = k(\boldsymbol{X}_*, \boldsymbol{X}_*)$. Note that since the number of training points need not equal the number of test points, then n has not attributable relationship with n^* and we cannot state any properties of the matrix K_*, much less say that it is a square matrix.

Since we have formally defined the distribution of $\boldsymbol{f}$ and we know that $\boldsymbol{f}_*$ is the same distribution but considering a different set of data points $\boldsymbol{X}_* = [x_1^*, \ldots, x_n^*]$, we can define the joint distribution of these vectors as

$$\begin{bmatrix} \boldsymbol{f} \\ \boldsymbol{f}_* \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \boldsymbol{0} \\ \boldsymbol{0} \end{bmatrix}, \begin{bmatrix} K & K_* \\ K_*^T & K_{**} \end{bmatrix} \right), \tag{1.2}$$

which is the joint probability distribution $\boldsymbol{f}, \boldsymbol{f}_*|\boldsymbol{X}, \boldsymbol{X}_*$. This is a massive step in constructing the regression model but we are still missing one component. When dealing with regression and prediction of values outside of the original training dataset, we do not need the joint probability, but the conditional probability of the test set given everything else we know.

Formally, this means that we need to find the probability distribution of $\mathbb{P}(\boldsymbol{f}_*|\boldsymbol{X}, \boldsymbol{X}_*, \boldsymbol{f})$. Luckily, since we are dealing with Gaussian distributions, there is a very neat closed form expression that describes this probability. Referring to section $A.2$ of [Rasmussen and Williams, 2006], and noting that the mean for both $\boldsymbol{f}$ and $\boldsymbol{f}_*$ is 0, then

$$\boldsymbol{f}_*|\boldsymbol{f} \sim \mathcal{N}(K_*^T K^{-1}\boldsymbol{f}, K_{**} - K_*^T K^{-1} K_*).$$

This probability function is what is used to calculate out of sample predictions for the out of sample data $\boldsymbol{X}_*$. Concretely, the mean of this conditional distribution, $K_*^T K^{-1}\boldsymbol{f}$, is the point prediction we assign to $\boldsymbol{f}_*$ for each test point in $\boldsymbol{X}_*$. The intuition here is fairly straightforward, $K^{-1}\boldsymbol{f}$ can be thought of as a set of weights learned from the training data, and K_*^T projects those weights onto the test points by measuring how similar each test point is to each training point through the kernel k. Points in $\boldsymbol{X}_*$ that are close to training points in $\boldsymbol{X}$ will borrow more heavily from nearby observed function values, while points far from all training data will have their predictions pulled back toward the prior mean of zero.

Equally important is the second component, the covariance $K_{**} - K_*^T K^{-1} K_*$. This matrix tells us how uncertain we are about our predictions: K_{**} is the prior covariance between test points before seeing any data, and $K_*^T K^{-1} K_*$ is how much of that uncertainty we can explain away given the training observations. The diagonal entries of this matrix give the predictive variance at each test point, which we can use to construct credible intervals around our predictions. This is one of the more appealing properties of Gaussian process regression, that we get a full probability distribution over predictions rather than just point estimates.

Gaussian Process Regression Over Noisy Observations

When there is inherent randomness or noise in the signals being calculated, the structure of the problem itself changes. If we can assume that the noisy signals we are receiving are of the form $y = f(\boldsymbol{x}) + \varepsilon$ with the noise or inherent randomness captured by ε and these being independent, identically distributed observations of a normal or Gaussian distribution, the covariance matrix of the observations therefore becomes

$$\text{cov}(\boldsymbol{y}) = \sum \boldsymbol{y} = K(\boldsymbol{X}, \boldsymbol{X}) + \sigma^2 I,$$

with I the identity matrix. In practical terms this means that the inherent variance of every observation is present as an added term on the diagonal of the covariance matrix (this is because of the assumption that the noise is

independent). This change causes the joint distribution originally defined in equation 1.2 to

$$\begin{bmatrix} \boldsymbol{y} \\ \boldsymbol{f}_* \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} K + \sigma^2 I & K_* \\ K_*^T & K_{**} \end{bmatrix} \right),$$

and the conditional distributions for prediction are therefore

$$\begin{aligned} \boldsymbol{f}_* | \boldsymbol{X}, \boldsymbol{y}, \boldsymbol{X}_* &\sim \mathcal{N}(\bar{\boldsymbol{f}}_*, \text{cov}(\boldsymbol{f}_*)), \text{ where} \\ \bar{\boldsymbol{f}}_* &:= \mathbb{E}[\boldsymbol{f}_* | \boldsymbol{X}, \boldsymbol{y}, \boldsymbol{X}_*] = K_*^T [K + \sigma^2 I]^{-1} \boldsymbol{y} \\ \text{cov}(\bar{\boldsymbol{f}}_*) &= K_{**} - K_*^T [K + \sigma^2 I]^{-1} K_*. \end{aligned}$$

These equations are for the entire training and test set though. For a test point x_*, we can define the vector $\boldsymbol{k}_*$ as the vector of covariances between this new test point and each of the points in the training dataset. This would be equivalent to taking the column associated with that single point from the test data matrix K_*. If we wanted just the closed form for this single out of sample observation, that would be

$$\begin{aligned} \bar{f}_* &= \boldsymbol{k}_*^T (K + \sigma^2 I)^{-1} \boldsymbol{y}, \\ \mathbb{V}(\bar{f}_*) &= k(x_*, x_*) - \boldsymbol{k}_*^T (K + \sigma^2 I)^{-1} \boldsymbol{k}_*. \end{aligned}$$

1.5 Summary and Outlook

We have covered a lot of ground in this chapter, moving from the discrete steps of Markov chains to the continuous, infinite-dimensional world of Gaussian processes. While these might seem like separate branches of probability, they provide the two pillars that support the rest of this work. The ergodic properties of the Markov chains we established are what will eventually allow us to actually run the BASS models through sampling based inference.

Gaussian process regression on the other hand gives us a way to turn a set of noisy observations into a full distribution of functions. As we have seen, the real power of the GP framework is that it doesn't just give us a single solution, rather it gives us a mathematically rigorous way to quantify what we don't know through the predictive variance.

However, having a good model is only half the battle. If we are dealing with a function that is expensive to evaluate, simply knowing its shape isn't enough; we need a plan. We need to know exactly where to look next to find the optimum as efficiently as possible. In the next chapter, we will take the posterior distributions we built here and use them as the engine for Bayesian optimization, moving from just modelling the function to making informed, sequential decisions about where we should evaluate it.

Chapter 2

Bayesian Optimization

This chapter (and the rest of this work) borrows heavily from the book [Garnett, 2023] as a source of consistent terminology. This book might be the most approachable and complete treatment of Bayesian optimization the author has found. We rely on it for standardizing notation and for some of the proofs and concepts, as well as for a general sketch of how this chapter is set out.

2.1 Introduction

Bayesian optimization is central to the experimental framework developed in this work. Understanding its motivation, assumptions, and mechanics is therefore essential before proceeding, and this section aims to provide a clear and self-contained foundation. The chapter that follows prioritizes conceptual clarity over generality and formal definitions with algorithmic details follow in subsequent chapters.

The necessity for Bayesian optimization arises from a particular kind of constraint, one that is distinct from the structural constraints we typically encounter in classical optimization. In traditional optimization, constraints govern the *form* of the solution space: for instance, in integer programming, the objective function is defined only over discrete inputs, or we restrict our interest to solutions that are integer-valued. In these settings, the primary objective is to locate an optimum of some objective function f over a well-defined domain $\mathcal{X}$. The computational cost of doing so, measured in the number of function evaluations required, is a secondary concern[1].

[1]Sometimes this is not even a concern. We evaluate these functions in terms of the amout of wall-clock time it takes to converge to a solution, not necessarily the amount of computations required to arrive at that solution.

Bayesian optimization shares this goal but introduces a critical additional assumption: the objective function f is *expensive to evaluate*. That is, each query $f(\mathbf{x})$ carries a significant cost: computational, financial, physical, or temporal. In this scenario, exhaustive or gradient-based search over $\mathcal{X}$ becomes impractical. Under this paradigm minimizing the number of evaluations is no longer a secondary concern but a core objective in its own right. We therefore seek not only to find the optimum of f, but to do so in as few evaluations as possible.

To make this concrete, consider the problem of hyperparameter optimization in machine learning, which will become directly relevant to this work in later chapters. When training a neural network, one must specify a learning rate η which is the rate at which the model updates its weights in response to the gradient of the loss. If η is set too high, training may become unstable and fail to converge. If it is set too low, convergence will be prohibitively slow or the model may stagnate at a poor local minimum. Evaluating a single candidate value of η requires a full training run, which for modern large-scale models can take hours or days. Exhaustive grid search or random search over the hyperparameter space is therefore wasteful and sometimes impossible. A more principled approach is needed.

Bayesian optimization provides exactly this. Rather than treating f as a black box to be sampled blindly, we maintain a probabilistic *surrogate model* that encodes our current beliefs about the shape of f given the evaluations observed so far. At each iteration, an *acquisition function* uses this surrogate to identify the point in $\mathcal{X}$ that is most worth evaluating next, balancing the competing demands of *exploitation* (querying near known good regions) and *exploration* (querying in regions of high uncertainty). This loop composed of update the surrogate, optimize the acquisition function, evaluate f, and finally start again, continues until a convergence criterion is met or a budget of evaluations is exhausted.

There are many domains beyond hyperparameter tuning where this paradigm is natural and necessary: experimental design in the physical sciences, A/B testing in product development, materials discovery, and the calibration of expensive simulation models, among others. What these problems share is the same fundamental structure: an objective value or function that is cheap to specify but costly to evaluate, with no available gradient information and a need to reach a good solution within a limited query budget.

The remainder of this chapter develops the mathematical machinery required to make this framework precise. We begin with Gaussian processes as the surrogate model of choice, before turning to the construction and properties of standard acquisition functions, and finally to the full Bayesian optimization algorithm and its convergence behavior. As the models that

we build and the problems we solve in general become more and more complicated, there has been a growing interest in this area of statistics as a solution to generating viable extrema candidates of expensive to evaluate functions. There has been extensive work on this field recently and in the past few decades, with examples such as [Bergstra et al., 2013], [Balandat et al., 2020], and [Akiba et al., 2019] that have impulsed this filed considerably. Hyperparameter optimization has recently been talked about as the "killer app" for Bayesian optimization, which revitalized interest in Gaussian Processes in the context of machine learning and Bayesian optimization in general. The interest in the field has lead to a great amount of interest and therefore research and publications tackling the method and providing theoretical backing to it.

In this chapter we will show the basic theoretical structure of Bayesian optimization, the three central components that make it up, and the decisions we have to make to be able to approximate these functions from a Bayesian point of view.

2.2 Components of BO

Before describing the components of Bayesian optimization in detail, it is worth being precise about the class of functions we are concerned with. A function $f : \mathcal{X} \to \mathbb{R}$ is said to be a *black-box function* if its internal structure is either unknown or computationally infallible to observe directly. More precisely, we have no access to an analytical expression for f, no gradient information $\nabla f(\mathbf{x})$, and no closed-form characterization of its global behavior. The only information available to us is the result of directly querying the function: given an input $\mathbf{x} \in \mathcal{X}$, we may observe the output $f(\mathbf{x})$. This stands in contrast to the functions typically encountered in classical optimization, where gradient-based methods can exploit the analytic structure of f to navigate the search space efficiently. In the black-box setting, no such structure is available, and evaluations themselves are the sole source of information about f.

Bayesian optimization is a sequential model-based approach to optimizing black-box functions that are expensive to evaluate. The goal is to find in some domain $\mathcal{X}$ a value x that maximizes the objective function f with the least possible number of evaluations. Practically, this means evaluating the value $f(x)$ for the least amount of times possible. One important distinction from other areas of optimization is that we do not impose any restrictions on the function f, except continuity[2]. This is notably different from many classical

[2]As we will see in later chapters, the entire point of this work will be relaxing this

settings. In integer optimization for instance, the domain $\mathcal{X}$ is restricted to $\mathbb{Z}^d$, meaning the objective function is only defined (or only of interest) at discrete points, and the geometry of the search space changes fundamentally as a result. At the other extreme are methods such as the finite element method (FEM) used to solve partial differential equations numerically. They require not only that the functions involved be continuous, but that they belong to appropriate Sobolev spaces, possessing weak derivatives of sufficient order to guarantee the existence and stability of solutions. Bayesian optimization asks for neither: f need not be discrete, nor differentiable, nor analytically specified in any way except continuity. Formally, let f be the objective function that we want to maximize, and let $\mathcal{D}$ be the set of observations we have made so far, which consists of pairs (x_i, y_i). We want to find the input x that maximizes the function $f(x)$, i.e., $x^* = \arg\max_{\mathcal{X}} f(x) = \arg\max_{\mathcal{X}} y$

The Bayesian optimization algorithm consists of three main components:

1. a prior distribution over the objective function,

2. an acquisition function,

3. and a method for updating the prior distribution based on the observed data.

The way this model works can be thought of as the following three steps:

1. Determining a probabilistic model that captures the observed behaviour of f, either by placing a prior distribution over f before any data has been observed, in the case of the first iteration, or by conditioning on accumulated data points $(x_i, f(x_i))$ for later iterations. We call this probabilistic model of f the prior, since it is used as one under this scheme[3].

2. Optimizing the selected acquisition function which is a criterion, defined formally below, that quantifies the utility of evaluating f at a given point that probabilistically estimates the best possible points for evaluation in the following iteration.

restriction. In practice the restriction enters not through the BO framework itself but through the Gaussian-process surrogate usually used to instantiate it, whose kernel assumes continuous numeric inputs; replacing that surrogate with BASS is what lets the same framework reach categorical and mixed search spaces.

[3] Another common way to jump-start this process is to begin by evaluating some random points in $\mathcal{X}$ and continuing the process from there.

3. Evaluating the function at the best candidate for improvement, therefore getting new data and updating our probabilistic model of the behaviour of the function.

Each of these three steps is treated in detail in the sections that follow.

The prior distribution over the objective function $f(x)$ is usually assumed to be a Gaussian process, but that need not be the case. While these type of models are beyond the scope of this work, as examples, we refer to [Shah et al., 2013] for the prior set to a t-Student distribution and [Wang et al., 2022] for a case study of the Poisson prior. Given the set of observations $\mathcal{D}$, the selected prior distribution defines a posterior distribution $p(f \mid \mathcal{D})$, which is our updated belief over the objective function given the data observed so far, that takes into account the uncertainty in the observations.

The acquisition function is a criterion that measures the expected utility of each point in the input space x for the purpose of optimization. The most commonly used acquisition function is the Expected Improvement (EI), which measures the expected improvement in the objective function if we were to evaluate the function at a particular input x.

$$EI(x) = \mathbb{E}[\max(f(x) - f(x_{\text{best}}), 0)],$$

where x_{best} is the current best input found so far. The idea is to select the input x that maximizes the EI, which balances exploration (trying new inputs) and exploitation (focusing on inputs that are likely to be good). This balance is controlled by hyperparameters, so that is another layer of complexity that needs to be controlled. In a sense, what we are doing is deferring solving the main optimization problem by generating a simpler to solve optimization problem in finding the relevant extrema of the acquisition function, thus allowing us to spend as little function evaluations as possible on evaluating the target function f. Much like the case with the prior distributions, there are other choices of acquisition function such as Thompson Sampling (TS) and Upper Confidence Bound (UCB), references for each can be found in [Kaufmann et al., 2012b] and [Kaufmann et al., 2012a] respectively.

The final component of BO is the method for updating the prior distribution based on the observed data. This is done under the overarching philosophy of Bayesian inference, which allows us to compute the posterior distribution over the objective function given the prior distribution and the observed data. The posterior distribution is then used as the prior distribution for the next iteration of the algorithm. What we wish to do is infer the value $\phi = f(x)$ for some new point in the domain $\mathcal{X}$.

2.3 Bayesian Formalization of the Update Step

The update step in Bayesian optimization follows directly from standard Bayesian inference. We briefly restate the relevant framework here in the notation established above, as it underpins the iterative structure of the algorithm.

This process begins by setting the prior distribution $p(\phi \mid x)$, which is our belief over possible values for ϕ before observation of the unknown x.

Assuming we observe the objective function at x and measure y, our optimization model supposes that the distribution of this measurement is determined by the value of interest ϕ through the observation model $p(y \mid x, \phi)$. With this new value y we can update our beliefs and generate the posterior distribution through the use of Bayes' theorem:

$$p(\phi \mid x, y) = \frac{p(\phi \mid x) \times p(y \mid x, \phi)}{p(y \mid x)}.$$

As with all applications of Bayes' theorem such as this one, the denominator is simply a constant term used to scale the probability. This means that we can simplify the expression by only keeping the kernel of the distribution and dropping the normalizing term, and using the proportionality operator $\propto$, as follows.

$$p(\phi|x, y) \propto p(\phi|x) \times p(y|x, \phi).$$

In the Bayesian framework, the posterior distribution is often not the final goal but a starting point for further tasks such as prediction or decision-making. This builds on the iterative process under which we are working in Bayesian optimization, where new information is added to the approximation of the function every time a function evaluation happens. As an example, consider the case where we want to predict the outcome of an independent, repeated noisy observation at x, y'. We can treat the outcome as a random variable and derive its distribution by integrating our posterior belief about ϕ against the observation model:

$$p(y'|x, \mathcal{D}) = \int p(\phi|x, \mathcal{D}) \cdot p\left(y'|x, \phi\right) \, d\phi.$$

This equation represents the posterior predictive distribution for y' given we know x and a set of data $\mathcal{D}$. By integrating over all possible values of ϕ, weighted by their plausibility, the posterior predictive distribution naturally accounts for uncertainty in the unknown objective function value.

$$p(y'|x, \mathcal{D}) = \int p(\phi|x, \mathcal{D}) \cdot p\left(y'|x, \phi\right) \, d\phi.$$

This equation represents the posterior predictive distribution for y' given we know x and a set of data $\mathcal{D}$. By integrating over all possible values of ϕ, weighted by their plausibility, the posterior predictive distribution naturally accounts for uncertainty in the unknown objective function value.

Taken together, the three components described in this section define the Bayesian optimization loop. Beginning with a prior $p(\phi \mid x)$ encoding our initial beliefs about the objective, we select the next query point by maximizing the acquisition function derived from the current posterior predictive. We then evaluate f at that point, observe y, and apply Bayes' theorem to update our beliefs, yielding a new posterior that serves as the prior for the next iteration. This cycle repeats until a stopping criterion is met, typically a budget of function evaluations. The efficiency of the entire procedure depends critically on the quality of the surrogate model used to represent $p(\phi \mid x, \mathcal{D})$.

From here is where the link to Gaussian Processes lies. Gaussian Processes serve as a natural and particularly convenient choice of surrogate, providing a flexible, non-parametric prior $p(f)$ over functions whose posterior predictive distributions are analytically tractable. It is this closed-form tractability that makes GPs the dominant choice in Bayesian optimization: the acquisition function can be computed and maximized efficiently at each iteration without approximation. The following section develops the theory of Gaussian Processes in detail.

Chapter 3

Sequential Model Based Optimization

There has been a lot of work done over the last few years in the space of Bayesian Optimization. One primary advance has been the formalization of sequential model based optimization (SMBO) as a general framework for Bayesian Optimization. The general idea of these types of models is to divide the optimization problem into two stages, first fitting a model from a specific class that predicts the performance on the target function and then gathering additional data to further refine the model that is used as a surrogate for the actual function we wish to evaluate. As detailed in [Koehrsen, 2018], the two steps can be broken down into this pseudo-algorithm for the problem of finding the best hyperparameters for a generic machine learning model:

1. Build a surrogate probability model for the target function
2. Find the hyperparameters that perform the best on the surrogate
3. Apply these hyperparameters to the true objective function[1]
4. Update the surrogate model incorporating the new results
5. Repeat steps 2-4 until a limit is reached (maximum number of iterations, time spent, or others such as cost).

While all SMBO algorithms follow these basic steps, the true power of it is in the flexibility it provides when selecting the surrogate model and when selecting how the new results can be incorporated into the model itself (steps

[1]This is the part that is expensive to evaluate, ideally, we want to minimize the amount of times that we actually run the true objective function.

3 and 4). This flexibility is the main point of the thesis, so it is worth dwelling on. Nothing in the SMBO template requires the surrogate to be a Gaussian process; the template asks only for a model that predicts the objective and carries some notion of uncertainty about its predictions, and any model meeting that description can be slotted in. This matters because the surrogate is not a neutral choice: it silently determines which problems the optimizer can even represent. A Gaussian process, the usual default, builds its predictions from a kernel that measures the numeric distance between inputs, and so it presumes a continuous search space. Swap the surrogate for one that does not make that presumption, and the same SMBO loop suddenly applies to problems the GP cannot natively express, most notably problems with categorical inputs. That substitution is exactly what this thesis carries out: it keeps the SMBO framework fixed and replaces the surrogate with a Bayesian Adaptive Spline Surface, whose treatment of categorical variables is native rather than bolted on. The acquisition step (the "how to incorporate new results" half of the flexibility) is held fixed across the methods we compare, precisely so that the effect of changing the surrogate can be isolated.

As is clear by the title of this work, the main contribution is the implementation of the BASS model as a surrogate model under this framework, but there have been a large amount of different methods and models used as surrogate models as well as a large amount of ways of incorporating the results into the surrogate, known as the selection function or evaluation criteria depending on the literature being read. The next sections detail the other models that are commonly used as surrogate models, and where these models got their roots.

Chapter 4

Surrogate Models for SMBO

4.1 Introduction

4.2 A Survey of Existing Surrogate Models

Different surrogate models can capture different aspects of the underlying function, each with its own strengths and weaknesses. The most common choice is Gaussian Processes (GPs). The selection depends on factors such as the dimensionality of the problem, the smoothness of the objective function, its domain (fully continuous, hybrid or discrete), and the available computational resources. In this section, we provide some texture as to the common surrogate models for the framework.

4.2.1 Gaussian Processes

While much time has already been dedicated to GPs in this work, it is worth noting that the strengths that define this method merit it. First and foremost, the models provide a robust probabilistic framework under which to model the underlying function. They not only predict the mean of the function but also provide a measure of uncertainty baked in (variance and covariance) associated with the predictions outside of the already estimated points. This uncertainty estimation is what allows Bayesian Optimization to work, since the selection of the next best point to evaluate is generally the point that we have the most uncertainty about, coupled with it being a proper candidate for a minima or maxima, depending on the situation. The choice between selecting candidates based on uncertainty or chances of being a better choice than the already explored points give us a trade-off in that we can tune depending on what we find most relevant for the problem by

adjusting the models hyperparameters.

While these models do not work well for combinatorial or categorical domains[1] because these models explicitly assume a continuous search space, if the problem has a relatively low dimension (not uncommon for hyperparameter optimization), then it is very efficient computationally to calculate. It is worth being precise about why the continuous assumption is so binding, because it is the single most important point of contrast with the surrogate this work proposes. A Gaussian process is defined entirely through its kernel $k(x, x')$, and the standard kernels (for example the squared exponential $k(x, x') = \sigma_f^2 \exp(-\ell(x - x')^2/2)$ discussed in the previous chapter) are functions of the numeric distance $(x - x')$ between inputs in $\mathbb{R}^d$. A categorical variable, whose levels are unordered labels with no meaningful distance between them, simply has no place in such an expression. In practice one is forced to encode categories as numbers, which fabricates an ordering that the problem does not have, or as one-hot indicators, which distorts the geometry the kernel relies on, and either choice degrades the surrogate. This is not to say GPs are fundamentally incapable of handling categorical inputs: a substantial line of work extends them to discrete and mixed spaces through specialized kernels, latent embeddings, or continuous relaxations [Garrido-Merchán and Hernández-Lobato, 2020; Ru et al., 2020; Zhang et al., 2020; Wan et al., 2021a]. The point is rather that this capability is *additional machinery*, not a property of the standard GP, and the existence of this whole research line is itself evidence that the default model cannot do it. There is also the matter of kernel selection, which gives the user unparalleled flexibility in continuous domains, since the kernel can be adapted to predict in a family of functions (such as strictly linear ones) if there is information known about the behaviour of the objective function already known.

Another advantage that cannot be overstated is the interpretability of these Gaussian Processes. These models not only provide mean value calculations, but also confidence intervals. This makes it easier to explain why the model is making certain choices, and with most types of problems, it is generally preferable to start with a simple and understandable method and scale up if necessary than to use a highly complex method which might become its own black box. Because of the nature of these models, they also tend to work well with small amounts of data, which goes hand in hand nicely with the restriction that we set off with for Bayesian Optimization, that being that the objective function is expensive to evaluate. This becomes a problem when dealing with surrogate models such as neural networks, which require

[1] This is a very loaded statement, but the authors think it is correct. There is also literature to back this up, such as [Wang et al., 2023] §3.2.

orders of magnitude more data to make proper predictions.

Yet another large benefit worth discussing is the fact that these models interpolate. The black box objective functions that we are assigning uncertainties to and calculating pointwise values for are at the completely deterministic. What the framework of Bayesian Optimization allows us to do is assign that uncertainty based on our own standing relative to the function, this is, the functions themselves are not probabilistic or uncertain, but our understanding of them is. Other surrogate models such as BASS which we are proposing do not interpolate perfectly, in the case of BASS because the estimation is done through MCMC which generates a probability distribution over a point in this use case. Because of their underlying construction and properties Gaussian Processes not only interpolate with zero variance in the points already calculated, but assign a low variance to regions of the domain close to the ones that have been calculated, and as such reduce the need for unnecessary exploration which is important under the framework that the objective functions are costly to evaluate.

To conclude, it is important to note that while all of these theoretical benefits are incredibly important, it is also worth mentioning that there is a lot of work in existing libraries and frameworks that has already been done to facilitate the use of GPs in the context of Bayesian Optimization. Because it is the de-facto standard in the area, most all of the existing packages for SMBO and BO specifically include modules and tools to define a Gaussian Process as the surrogate model. Practical inertia must not be discounted as a critical reason to explain the prevalence of GPs in this field.

4.2.2 Tree Parzen Estimators

The inception of this algorithm comes from the paper [Bergstra et al., 2011], which provides a thorough explanation of the mechanisms that define TPEs (Tree Parzen Estimators). Even though this algorithm was created in 2013, it is still considered one of the premier algorithms for hyperparameter tuning using BO. This algorithm excels at problems with high dimensionality and small fitness evaluation budgets. The success of TPEs lies in their ability to efficiently explore the hyperparameter space by modelling the objective function using a combination of tree-based estimators and probabilistic density estimation techniques, such as Parzen estimators (also referred to as kernel density estimators), which entail a straightforward averaging of kernels centered around existing data points. Probably the most interesting aspect of this surrogate model is that while we normally are interested in calculating $\mathbb{P}(y|x)$ which is the probability of an objective function given the points already calculated, TPE takes a different approach and calculates

$\mathbb{P}(x|y)$, which is the probability of having the points already calculated given a certain objective function being the "correct" one.

By iteratively updating a probabilistic model of the objective function and using it to guide the search for promising hyperparameter configurations, TPEs strike a balance between exploration and exploitation, allowing it to converge to near-optimal solutions with relatively few evaluations. One large benefit to TPE models is that there is only one hyperparameter to tune, namely $\gamma \in [0, 1]$, which controls what we consider to be a good function evaluation and what we consider to be a bad one. Going into mode detail than this would require a section onto itself, but more details can be found in [Bergstra et al., 2011], in the recent and very readable account of its algorithm components in [Watanabe, 2023], and in the paper for Hyperopt, which is the python library published by the creators of the model for hyperparameter tuning in python [Bergstra et al., 2013]. One important thing to consider is that these types of models require more data to achieve great results when compared to GPs in general, and as such, if the function is prohibitively expensive, it could become a problem.

A property of TPE that is particularly relevant to this work is that, because it estimates a density per hyperparameter rather than a kernel over a joint continuous space, it handles categorical inputs natively: for a categorical variable it simply estimates how its levels distribute among the good and the bad trials. In this respect it sits on the same side as BASS and opposite the Gaussian process, which is why TPE has become a default for the mixed and categorical hyperparameter spaces that arise in practice. For exactly this reason we later include TPE, in its reference Optuna implementation [Akiba et al., 2019], as a baseline in the experiments, so that BASS-BO is measured not only against a Gaussian process on the continuous problems but against a purpose-built categorical optimizer on the categorical ones.

4.2.3 Bayesian Neural Networks

Bayesian neural networks (BNNs) are a type of neural network that incorporates Bayesian methods to model uncertainty in the network's weights and predictions. Traditional neural networks typically output a point estimate for each input, representing a deterministic prediction. In contrast, BNNs provide a probabilistic framework that quantifies uncertainty associated with predictions. Bayesian neural networks, with their probabilistic framework, provide a natural way to model the objective function and its uncertainty. During the optimization process, BNNs can be trained on the observed data points (input-output pairs), and the resulting posterior distribution over the weights can be used to make predictions about the objective function's be-

haviour in unobserved regions of the input space.

Furthermore, BNNs offer the advantage of not only providing point estimates of the objective function but also quantifying the uncertainty associated with those estimates. This uncertainty estimation is crucial in BO for guiding the exploration-exploitation trade-off.

This area of research is thriving, and many different neural network architectures have been proposed to further optimize the BO model. This work does not delve into any particular methodology, but a run-down of recent advances and their particular use cases (e.g., large scale data, high dimensional BO, combinatorial optimization) can be seen in [Wang et al., 2023].

4.2.4 Random Forest, Gradient Boosted Machines

Tree and forest based algorithms have received a lot of praise in recent years. They are unreasonably effective when dealing with tabular data in a traditional setting, even outperforming deep neural networks which seem to have overtaken these models in everything but tabular data [Grinsztajn et al., 2022]. In the context of BO, they can be good surrogate models, as they are generally good regressors. They are highly flexible, as they can automatically handle nonlinearities, interactions, and high-dimensional feature spaces without the need for explicit feature engineering. They are also highly scalable, as they can handle a wide range of problem sizes and complexities, making them suitable for BO tasks with varying levels of computational resources.

While these models may seem like a natural choice, there are some limitations in selecting them as surrogate models, most relevantly, the lack of probabilistic outputs. Unlike Bayesian Neural Networks (BNNs) or GPs, RFs and GBMs do not naturally provide probabilistic outputs or uncertainty estimates. This makes it more challenging to quantify uncertainty in predictions, which is crucial for guiding the exploration-exploitation trade-off in BO. This is lessened by the fact that they can indirectly capture uncertainty through techniques such as bootstrapping and out-of-bag estimation, and as such, these methods can be leveraged to estimate the variability in predictions and guide the exploration-exploitation trade-off in BO. There are also some other limitations, like the fact that these models are basically black boxes in and of themselves if the problem is of a high dimension or there are many interactions between variables, or that if we are not careful with regularization, they can be overfit very easily. One other large problem is simply the amount of hyperparameters these models have, as tuning them to perform well on a BO problem is not an easy task, and changes between one objective function and another can make previous hyperparameter configurations unusable, or lead to model stagnation, which is not something we want with objective

functions that are expensive to evaluate.

A practical advantage that cannot be overlooked is the sheer amount of libraries and support that exists for these types of models. They have been studied intensely for the last decades, and the library implementations that exist in popular programming languages are some of the best designed Machine Learning libraries out there. Still, the choice of a RF or GBM must be done carefully, as heavy trade-offs could hurt more than help in the long run.

4.3 Other Techniques for Optimizing Hyperparameters

4.3.1 Grid Search

In grid search, the hyperparameter space is divided into a grid, and models are evaluated at each point in the grid. While grid search is exhaustive and guaranteed to find the optimal solution within the search space. The largest trade-off is that this method of search is expensive, since each function evaluation is defined by the problem to be expensive. If this is not a problem, then this is one of the best methods out there. The constraint of expensive to evaluate black-box functions is what necessitates the rise of other search algorithms, such as Bayesian Optimization.

4.3.2 Random Search

This is a simple technique where hyperparameters are randomly selected from a predefined search space. Despite its simplicity, random search can often be surprisingly effective and computationally efficient. One of the main problems comes when the search space becomes complex, because of the curse of dimensionality, using a random search strategy could lead to a very narrow search in terms of the space as a whole. This can be compounded by not looking at points "close" to the ones where we already found good results, which can lead to us not finding local or global maximums or minimums, and instead spending valuable time looking at regions we might already know to probably not yield interesting results. A good reference for both grid search methods and random search can be found in [Bergstra and Bengio, 2012].

4.3.3 Genetic Algorithms

Part of the larger group composed of evolutionary algorithms, they are based on the idea of natural selection when evaluating certain points in the hyperparameter mesh that we define. To extend the analogy, the fittest (in this case the hyperparameter vectors with the lowest or highest values) are selected to product offspring. This is done iteratively, and these individuals traits (good guesses for the optimums) are passed on to the next generations. These algorithms are generally more complex than anything else covered in this work and as such we do not elaborate much further, but a good and recent reference is [Xiao et al., 2020], which describes these algorithms used to tune hyperparameters in deep learning models.

Chapter 5

BASS and their Origins

The main proposal in this work is the use of a BASS model as the surrogate model in the framework of Bayesian Optimization rather than a traditional Gaussian Process or any of the alternatives already discussed. To understand this method and why it is useful for this purpose, we develop first the theory that lead to the construction of this method, and then explain its behaviour in some detail.

As with most other algorithms used for regression tasks, BASS is built upon a rich history of work that was previously done in the field. It begins with CART and decision trees, which MARS models are based on, and in turn BMARS (Bayesian MARS) uses the same model form as MARS but builds the model using the traditional Bayesian approach of defining a prior and constructing from there. Finally, BASS is a specific case of the BMARS model. This is much more clear when viewed as a diagram, as shown in the following figure.

Figure 5.1: The theoretical basis for BASS.

5.1 Decision Trees

Decision trees are a fundamental and versatile tool that comes in two main flavours: classification trees and regression trees. These two types of trees serve distinct purposes and are employed to make predictions in different contexts. The term "Classification and Regression Tree" (CART) analysis serves as an umbrella term that encompasses both classification and regression tree procedures. CART was first introduced by Breiman et al. in 1984 [Breiman, 2017], and it represents a unified framework for decision tree modelling, allowing for flexibility in handling both classification and regression tasks within the same algorithm. While classification and regression trees share common principles, such as recursive binary splitting and impurity measures (e.g., Gini impurity for classification and mean squared error for regression), they also exhibit some differences in terms of the criteria used to determine the best feature and split point at each node. The final predictions for a new data point are obtained by traversing the tree from the root node to a leaf node and returning the average (for regression) of the target variable for the training samples in that leaf node.

Figure 5.2: A decision tree.

In classification tree analysis, the primary objective is to predict the discrete class or category to which a given data point belongs. The decision tree algorithm partitions the data into subsets based on input features and assigns each data point to a specific class by following a path through the tree.

On the other hand, regression tree analysis is employed when the predicted outcome is a continuous, real-valued number. Similar to classification trees, regression trees partition the data based on input features. However, in this context, the goal is to estimate a numeric value for each data point, typically by computing the average of the target variable within each leaf node.

In both cases, the decision tree serves as a transparent and interpretable model, making it valuable for exploratory data analysis and predictive modelling. The tree's structure allows for a straightforward understanding of how decisions are made, and predictions for new data points are obtained by navigating the tree from the root node to an appropriate leaf node, where the final prediction is based on the characteristics of the training samples within that leaf.

The versatility and comprehensibility of decision trees have contributed significantly to their widespread use in various fields, and the distinctions between classification and regression trees make them adaptable tools for addressing a wide range of data analysis and prediction tasks. One problem that these methods tend to have though is overfitting, since if we allow the model to keep generating new nodes or leafs, the tree ultimately becomes too complex and every data point that we would like to analyse is simply too narrow to be useful as an aggregation scheme.

While decision trees are useful on their own their real power comes from the versatility they provide, leading to a wide range of methods being born from the basic idea of splitting decisions into paths. These methods are sometimes called *ensemble* methods, because they involve the creation and combination of multiple models to form a more robust and accurate predictive model. As an interesting side note, the term ensemble is borrowed from the world of music and theatre. Instead of relying on a single predictive model, these models create an ensemble of multiple models, each of which may have its own strengths and weaknesses. These individual models, often referred to as base models or weak learners, are combined in a way that leverages their collective intelligence to make better predictions.

Ensemble methods work on the premise that combining diverse models can reduce the risk of overfitting, enhance generalization, and improve predictive accuracy. Each model in the ensemble may make errors on some portions of the data, but by aggregating their predictions, the ensemble can produce more accurate and stable results. Common ensemble methods include Random Forest, Gradient Boosting, Adaptive Boosting, Bagging, and many others. The difference in these models is how they combine these different decision trees to achieve a more stable model.

5.2 MARS Models

Now, let's shift our focus the following technique down the chain for us known as MARS, which stands for Multivariate Adaptive Regression Splines. MARS builds upon the foundation laid by decision trees but introduces a differ-

ent paradigm for modelling complex relationships within data. Rather than using piecewise constant approximations, MARS leverages piecewise linear functions, thereby extending the toolbox of non-linear regression methods.

MARS is particularly valuable when the relationships between input features and the target variable are not strictly linear but can be approximated as a series of linear segments. Let's delve into the mathematical foundations of MARS to gain a deeper understanding.

For the following section, we will adopt the terminology and follow the development presented in [Friedman, 1991a] and [Friedman, 1991b].

MARS is defined using functions that are piecewise linear in the form of $(x-t)+$ and $(t-x)+$. These functions take the maximum between 0 and the value within the function, as follows:

$$(x-t)_+ = \max\{0, x-t\} = \begin{cases} x-t & \text{if } x > t, \\ 0 & \text{if } x \leq t. \end{cases}$$

Each function is piecewise linear with a slope change at the value t, which makes them linear splines. Each pair divided at the value t is referred to as a "reflected pair". The idea of the method is to form reflected pairs for each input variable X_j with slope changes a selection of observed values x_{ij}. Following this construction and considering a variable X_j, the collection of basis functions is:

$$\mathcal{C} = \{(X_j - t)_+, (t - X_j)_+\}, \text{ with } t \in \{x_{1,j}, x_{2,j}, \ldots x_{N,j}\}, \text{ y } \ j = 1, 2, \ldots, p.$$

And each reflected pair appears as follows, and are shown in figure 5.3:

$$h_1(X) = h(X_j - x_{ij})$$

$$h_2(X) = h(x_{i,j} - X_j)$$

So, for each X_j, the set of candidate reflected pairs that exist at each of the x_{ij} points of that variable is shown in figure 5.4.

This means that if all input values are distinct, there are a total of $2Np$ basis functions, and $2N$ divisions in each of the splines for each variable. The model we use to combine all variables is an additive one in $\boldsymbol{\beta}$:

$$f(X) = \beta_0 + \sum_{m=1}^{M} \beta_m h_m(X),$$

where each function $h_m(X)$ is an element of $\mathcal{C}$ or a linear combination of these basis functions. For this initial explanation, we will use only functions that

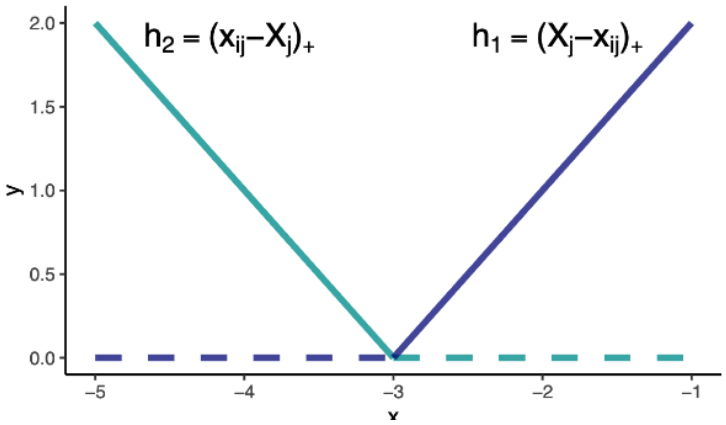

Figure 5.3: The constructed reflected pairs.

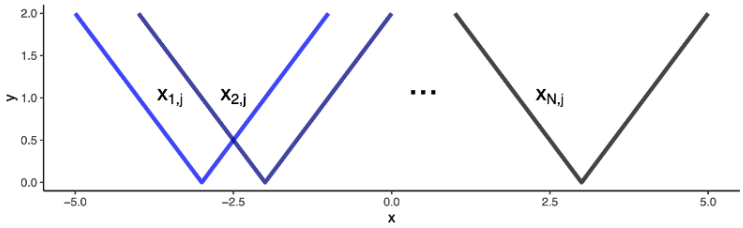

Figure 5.4: Collections of hinge functions.

do not involve interactions (the model with interactions works very similarly to CART decision trees).

The model construction process (denoted as $\mathcal{M}$) begins with the base model $\hat{f}(X) = \hat{\beta}_0 = h_0(X) = 1$, where a term functioning as an intercept at the origin is defined. To add terms incrementally, all functions in the set $\mathcal{C}$ are candidates for entry into $\mathcal{M}$. For each observation x_{ij}, the new model involving these functions is fitted as follows:

$$\hat{f}(X) = \hat{\beta}_0 + \hat{\beta}_1 h_1(X) + \hat{\beta}_2 h_2(X).$$

Since this is a linear form in terms of each of the functions h_i, the adjustment of each parameter $\hat{\beta}_i$ is done by minimizing the sum of squares.

The reflected pair that is added is the one that minimizes the training error. This process is repeated for each of the remaining reflected pairs, solving OLS to determine the new $\boldsymbol{\beta}$, and adding the one that continues to minimize the error. The stopping criterion can be either that none of the remaining reflected pairs reduces the error enough (in absolute or relative terms) or until we have a predetermined number of variables in the model.

This leads to overfitting the data, but this is the goal in this part of the procedure when minimizing the training error. Once we have $\mathcal{M}$, we start the second part of model fitting, which is the pruning part. In this case, terms $h_i(X)$ are removed iteratively, starting with the one that produces the smallest increase in the residual sum of squares when removed. This procedure yields a better model for each λ size, denoted as $\hat{f}_\lambda$.

Several procedures can be used to estimate the optimal value of λ, such as cross-validation or bootstrap, but this involves a significant computational cost. To minimize this cost, MARS models generally use a generalized cross-validation (GCV) procedure. This criterion is defined as:

$$GCV(\lambda) = \frac{\sum_{i=1}^{N}(y_i - \hat{f}_\lambda(x_i))^2}{\left(\frac{1-M(\lambda)}{N}\right)^2},$$

where the value $M(\lambda)$ is the number of effective parameters in the model, depending on the number of terms plus the number of breakpoints used, penalized by a factor (this factor is 2 in the case for additive in X that we are explaining, and 3 when there are interactions).

Now that we have described the base model, we can go back and consider the differences when incorporating interaction terms. Instead of just adding terms additively, products between the existing h_ℓ functions in the model and between each reflected pair of each x_{ij} can be added. Note that this general case encompasses the previous one because if you want to add a reflected pair without interacting with the other variables, you simply multiply by $h_0 = 1$.

In this way, a model $\mathcal{M}$ with M terms will be updated as follows after finding the combination $\hat{\beta}_{M+1}h_\ell(X) \cdot (X_j - t) + +\hat{\beta}M + 2h_\ell(X) \cdot (t - X_j)_+$ that best minimizes the training error:

$$\hat{f}(X) = \hat{\beta}_0 + \sum_{m=1}^{M} \hat{\beta}_m h_m(X) + \hat{\beta}_{M+1}h_\ell(X) \cdot (X_j - t)_+ + \hat{\beta}_{M+2}h_\ell(X) \cdot (t - X_j)_+,$$

where $h_\ell \in \mathcal{M}$ and $t = x_{ij}$ for any case.

One interesting consideration is that the added flexibility of these models does not end with it being piecewise linear, as one small alteration that can be made is to replace the truncated linear functions with truncated cubic basis functions or other functions of a higher order, which provides allows the resulting model to be fully differentiable within its domain.

In conclusion, our exploration of Multivariate Adaptive Regression Splines (MARS) is a powerful and versatile technique for modelling complex relationships within data. Unlike traditional linear regression, MARS leverages

piecewise linear functions, making it a valuable tool when dealing with non-linear relationships between input features and the target variable. The methodology behind MARS involves the creation of reflected pairs and the incremental addition of basis functions to the model, effectively adapting to the data's intricacies. The model construction process aims to minimize the training error by iteratively selecting the reflected pair that provides the most significant reduction in error. This leads to a model that can potentially overfit the data, but this is by design during the training phase. Subsequently, a pruning step is employed to refine the model, removing terms that do not contribute significantly to the model's predictive power.

5.3 BMARS Models

The key innovation that distinguishes BMARS from its predecessor lies in its Bayesian framework. While MARS primarily focuses on constructing piecewise linear models by iteratively selecting basis functions to minimize training error, BMARS introduces Bayesian methods to bring a probabilistic perspective into this process. This integration of Bayesian principles allows BMARS to provide not only point estimates of model parameters but also entire probability distributions, enabling a more comprehensive understanding of the uncertainty inherent in the modelling process.

As described in [Denison et al., 1998], which is the original paper where this method was proposed, the idea behind this model is to allow more flexibility in the traditional MARS model by looking at it from a Bayesian perspective. This is done by mimicking the MARS model with changes in framing, namely by considering the number of basis functions, along with their *type*, their coefficients and their form (the positions of the split points and the sign indicators) as random.

We take the definition of a basis type from the original paper and a small example from there to make the definition tangible, and include it here for completeness.

Definition 10. *We consider basis functions X_i and X_j to be of the same type if $[x(1,i),\ldots,x(J_i,i)]$ is identical to some permutation of $[x(1,j),\ldots,x(J_j,j)]$. Hence, with m predictor variables, there are N different types of basis functions, where N is given by:*

$$N = \sum_{i=1}^{m} \binom{m}{i} = 2^m - 1. \tag{5.1}$$

Note that the sum does not include the constant basis function term (for which i would equal 0 in (5.1)), as this basis X_1 is always the sole constant

basis function in the model, so it cannot be chosen as a candidate basis. Frequently, some maximum order of interaction I is assigned to the model, such that $J_i \leq I$ for $i = 1, \ldots, k$. In which case, the sum in (5.1) only runs from 1 up to I. We let $T_i \in \{1, 2, \ldots, N\}$ denote the type of basis function X_i; thus, T_i effectively tells us which predictor variables we are splitting on, i.e., what the values of $x(1, i), \ldots, x(J_i, i)$ are.

As an example, suppose we have a problem with just two predictors, i.e., $m = 2$. Then the types of basis functions that could be in the model (not including the constant one) are $[\pm(x_1 - *)]$ (say, type 1), $[\pm(x_2 - *)]$ (say, type 2), and $[\pm(x_1 - *)][\pm(x_2 - *)]$ (say, type 3). So for all those basis functions which split only on predictor x_2, their types T_i are all equal to 2.

The key insight that this framework of construction is based on is that any MARS model can be uniquely defined by the following:

- The number of basis functions k,
- the coefficients of the basis functions a_i,
- the type of basis functions T_i,
- the knot points associated with each interacting term t_{ji},
- the sign indicators associated with each interacting term s_{ji}.

and as such, a probability distribution over the possible MARS models can be constructed, enabling a more comprehensive understanding of the uncertainty inherent in the modelling process since the response is an entire probability distribution rather than a single model. This framework also provides the ability to do ANOVA and Sobol decompositions for sensitivity analysis. One important aspect of this framework for construction is that the number of parameters varies with the number of basis functions k, and as such, a traditional method such as Markov chain Monte Carlo cannot be used directly, as it assumes constant parameters. An extended version of MCMC is therefore used, called reversible jump MCMC (first introduced in [Green, 1995]), that allows for a variable number of parameters.

5.4 BASS Models

Bayesian Adaptive Spline Surface (BASS) models are an extension of the family of non parametric regression models of which multivariate adaptive regression splines (MARS) was the first developed in 1991 and is the most

basic [Friedman, 1991b]. MARS is an extension of linear models, where interactions between variables and non linearities are modelled automatically.

The extension of this model of interest is the BMARS model. As the authors state in [Denison et al., 1998], the aim of BMARS is to provide a Bayesian model that mimics the MARS procedure. This is done by considering the number of basis functions, along with their type, the coefficients and their form, random. Then, a suitable MCMC algorithm, in this case reversible jump monte carlo [Green, 1995], is used to calculate a suitable posterior for the problem. The great advantage to RJMCMC over other MCMC techniques is that the number of parameters can be variable, and as such, is a great basis for the type of analysis that is necessary for models in the MARS class.

BASS models are an adaptation of BMARS models that include some efficiency improvements for posterior sampling alongside parallel tempering for better posterior exploration [Francom and Sansó, 2020]. For our purposes though, the most significant improvement is that it allows the use of categorical variables alongside some continuous variables. This is incredibly useful in the context of hyper-parameter optimization, where there are generally some categorical variables such as polynomial degree, smoothing, etc. While we remain in the one dimensional case and this improvement is not necessarily useful, the multi dimensional case will benefit greatly from this improvement.

A detailed rundown of the model, its priors, and its construction in general can be seen in [Francom and Sansó, 2020], but, we include the definition of the model presented in that work for completeness.

Let y_i denote the dependent variable and $\boldsymbol{x}_i$ denote a vector of p independent variables, with $i = 1, \ldots, n$. Without loss of generality, let each independent variable be scaled to be between zero and one. We model y_i as

$$y_i = f(\boldsymbol{x}_i) + \epsilon_i, \;\; \epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

$$f(\boldsymbol{x}) = a_0 + \sum_{m=1}^{M} a_m B_m(\boldsymbol{x})$$

$$B_m(\boldsymbol{x}) = \prod_{k=1}^{K_m} g_{km}[s_{km}(\boldsymbol{x}_{v_{km}} - t_{km})]_+^{\alpha}$$

The only difference between this setup and that of MARS and BMARS is the inclusion of the constant g_{km} in each element of the tensor product. This term is $g_{km} = [s_{km}(\boldsymbol{x}_{v_{km}} - t_{km})]_+^{\alpha}$, where $s_{km} \in \{-1, 1\}$ is referred to as the sign, $t_{km} \in [0, 1]$ is a knot, v_{km} selects a variable, K_m is the degree

of interaction, and α determines the degree of the polynomial splines. This normalizes the basis functions so that the basis coefficients $a_1, \ldots, a_M$ are on the same scale, making computations more stable.

The hinge form $[s_{km}(\boldsymbol{x}_{v_{km}} - t_{km})]_+^\alpha$ above is the basis used for *continuous* predictors, and it makes the role of the knot t_{km} a numeric one: it places a slope change at a point along a real axis. This is precisely the construction that does not transfer to a categorical predictor, whose levels have no order and no notion of distance between them. The way BASS resolves this, inheriting the treatment of categorical variables from [Friedman, 1991b] and [Denison et al., 1998], is to use a different kind of factor in the tensor product when the selected variable $\boldsymbol{x}_{v_{km}}$ is categorical: instead of a hinge, the factor becomes an *indicator that the variable's level lies in a chosen subset of its levels*. Formally, if a categorical predictor takes values in a finite set of levels $\mathcal{L}$, the corresponding factor in $B_m(\boldsymbol{x})$ is $\mathbf{1}[\boldsymbol{x}_{v_{km}} \in S_{km}]$ for some subset $S_{km} \subseteq \mathcal{L}$. The subset S_{km} plays the role that the knot t_{km} plays for a continuous variable, and like every other element of the model (the number of basis functions, the variables they involve, the knots, and now the level subsets) it is treated as random and explored by the reversible-jump MCMC. Continuous and categorical factors can appear together in the same product B_m, so the model represents interactions between the two kinds of variable directly; in the `BASS` package the maximum order of such categorical interactions is controlled by the `maxInt.cat` argument, and a predictor is treated as categorical simply by passing it as a factor column [Francom and Sansó, 2020]. This is the mechanism behind the claim made above, and it is what makes BASS able to act as a surrogate over the mixed continuous–categorical search spaces that arise naturally in hyper-parameter optimization, without the encodings that a continuous-only surrogate would require.

5.5 BASS Models as Surrogates in Bayesian Optimization

With the discussions of the previous chapter and this one, we now have the tools to use the BASS model as a surrogate. There are a couple of decisions that we have to make though so that the model works properly, discussed in detail in this section. Namely, they are the starting points of the problem, the discretization of the function domain, and how we will handle uncertainty in points that have not yet been evaluated.

5.5.1 Starting Point for the Algorithm

One common problem with BO is the cold start problem. Basically, it is very difficult for the system to draw inference or make predictions at the beginning of iterations because it simply has no relevant information. Another way of saying this is that the prior in this case is uniform (or discrete uniform depending on the context) over the entire function domain, which is not good. A simple solution to this problem is to give the algorithm a set of starting points in the domain, this could be to either do a small grid search to start for a small number of iterations, or do a random search with the same objective. Having these few points, then the surrogate has enough information to properly decide a good next point to evaluate with the specific acquisition function selected. The next chapter concerns the experiments that were run to test the potency and viability of this model, and in it, different amounts of starting points are used. Needing the starting points is not something unique to BASS models though, as many other models (for example GPs) also have trouble with cold starts. For the experiments, we made sure to begin all of them on the same amount of already evaluated points, calculated at random.

5.5.2 Discretization of the Function Domain

As with any numerical problem, one important step in designing the algorithm to solve it is the discretization of continuous variables. This process involves dividing the continuous input space into discrete intervals or points, enabling the algorithm to operate efficiently within a finite computational framework. The choice of discretization scheme can significantly impact the performance and convergence properties of the optimization process.

One approach to discretization is to define a grid over the input space. This involves partitioning each dimension into a set of equally spaced points, forming a mesh that covers the entire domain. While grid-based discretization offers simplicity and uniformity, it will suffer from the curse of dimensionality, especially in high-dimensional spaces, where the number of grid points grows exponentially with dimensionality.

The way we decided to discretize the space was using Latin Hypercube Sampling (LHS) [McKay et al., 2000]. LHS is a statistical sampling method used to generate a set of diverse and representative samples from a multidimensional probability distribution. It is particularly useful in optimization, where obtaining a set of samples that cover the entire parameter space evenly and efficiently is crucial. This method is better for this purpose than a simple grid search or a traditional pseudo-random sample of points generated

through Monte Carlo methods. We do not go into too much detail into these methods as they are not the main point of this work, but the interested reader can look to the previously referenced [McKay et al., 2000] or [Stein, 1987].

5.5.3 Uncertainty Estimation

At first glance, selecting BASS as the surrogate seems to raise a problem when compared to GPs: a Gaussian process hands us a closed-form predictive mean and variance at any unobserved point, and that variance is exactly what an acquisition function needs in order to decide where to look next, the literal point of Bayesian Optimization. It would appear that a spline-regression surrogate gives us only a point prediction and no such uncertainty. This, however, is not the case, and the reason is that BASS is not a point estimator but a fully Bayesian model. The reversible-jump MCMC does not return a single fitted surface; it returns a sample from the posterior distribution *over surfaces*, that is, a collection of plausible spline models given the data observed so far. Evaluating these retained posterior draws at any candidate point $\boldsymbol{x}$ therefore yields a sample from the posterior predictive distribution of $f(\boldsymbol{x})$, and the spread of that sample is precisely the uncertainty we were worried about. Crucially, this uncertainty is native to the model rather than bolted on after the fact, and it does not assume the predictive distribution is Gaussian, which matters because the BASS posterior predictive is in general a skewed, multi-modal mixture of spline surfaces.

Given these draws, we score a candidate with the same acquisition principle used for the GP baseline, namely Expected Improvement, but computed by Monte Carlo over the posterior samples rather than in closed form. Writing f_{best} for the best (minimal) value observed so far and $\{f^{(s)}(\boldsymbol{x})\}_{s=1}^{S}$ for the S posterior predictive draws at $\boldsymbol{x}$, the Monte Carlo Expected Improvement is

$$\widehat{\mathrm{EI}}(\boldsymbol{x}) = \frac{1}{S}\sum_{s=1}^{S} \max\bigl(0,\ f_{\text{best}} - f^{(s)}(\boldsymbol{x})\bigr)\,,$$

which requires no exploration weight to tune: the exploration–exploitation balance is supplied automatically by the dispersion of the posterior draws. As a cheaper alternative we also implement Thompson sampling, which draws a single surface from the posterior and moves toward its minimizer. Both choices use BASS's own posterior as the source of uncertainty, and because the GP baseline is scored with the same Expected Improvement criterion, the two methods differ only in the surrogate, which is exactly the comparison this work is after. The details of how candidates are generated and scored each

iteration are given in the next chapter.

Chapter 6

Experimental Validation

6.1 Introduction

The previous chapters established the theoretical basis for Bayesian Optimization (BO) with BASS models as surrogates. We discussed initialization challenges, domain discretization, and uncertainty estimation, with special attention to unexplored regions of the search space. The next step is empirical validation.

This chapter presents the experimental framework used to evaluate BASS-based BO for hyperparameter optimization. The goal is to compare BASS against established alternatives and determine where it performs well, where it underperforms, and why. In particular, we study whether BASS can address practical concerns such as cold starts and sample efficiency on low-to-moderate-dimensional problems (two to six input dimensions), the regime in which expensive black-box optimization is most commonly deployed.

The methods compared in this chapter are Random Search, Gaussian Process-based BO (GP-BO), BASS-based BO (BASS-BO), and, on the categorical and mixed benchmarks, the Tree-structured Parzen Estimator (TPE). Experiments are conducted across synthetic benchmark functions, including both smooth continuous problems and categorical/mixed-variable problems, to provide a broad evaluation. The analysis focuses on both optimization quality and computational efficiency.

The chapter is organized as follows. First, we describe the experimental design, including methods, models, datasets, metrics, and statistical procedures. Next, we present and interpret the results. Finally, we summarize the main findings, identify limitations, and provide recommendations for future work.

6.2 Experimental Design

6.2.1 Objective and Scope

The primary objective is to evaluate BASS-BO against established hyperparameter optimization methods on two dimensions: optimization effectiveness and optimization efficiency.

This evaluation criterion is consistent with the central motivation of BO and Sequential Model-Based Optimization, namely to reduce expensive objective function calls while maintaining strong optimization performance. The study includes classical benchmark functions (functions from [Surjanovic and Bingham, 2024]) to test behavior under controlled and application-oriented conditions.

6.2.2 Hyperparameter Optimization Methods

The methods compared are the ones previously mentioned in this work along with the new framework with BASS as the surrogate:

- **BASS-Based Bayesian Optimization (BASS-BO)**: the proposed method. BASS surrogates model the response surface and uncertainty, and an acquisition function selects new evaluations iteratively.
- **Random Search**: random sampling from predefined hyperparameter distributions. It is often more efficient than Grid Search in moderate to high dimensions, as shown in [Bergstra and Bengio, 2012].
- **Gaussian Process-Based Bayesian Optimization (GP-BO)**: a standard BO baseline using Gaussian Processes for surrogate modeling and uncertainty-guided acquisition.
- **Tree-structured Parzen Estimator (TPE)**: a strong external baseline and, for our purposes, an especially relevant one. TPE is the de-facto standard for hyper-parameter optimization with mixed and categorical search spaces, and, like BASS but unlike a plain Gaussian process, it handles categorical inputs natively (it models separate densities over good and bad trials and, for a categorical variable, simply estimates a distribution over its levels) [Bergstra et al., 2011; Watanabe, 2023]. We use the reference implementation in the Optuna library [Akiba et al., 2019], driven from R so that it runs under exactly the same protocol as the other methods. Including it lets us ask not only whether BASS-BO beats a Gaussian process on categorical problems,

where the Gaussian process is at a structural disadvantage, but how it compares to a method that is itself designed for them.

The dataset in particular we will be using is the Virtual Library of Simulation Experiments: Test Functions and Datasets [Surjanovic and Bingham, 2024]. This suite is composed of standard functions such as the Branin-Hoo function, commonly used to test Bayesian Optimization problems.

6.2.3 Experimental Procedure

Each run follows a common protocol to ensure fair comparison.

1) Initialization All methods start from the same initial design of $n_0 = \max(2d+1, 8)$ points, generated by maximin Latin Hypercube Sampling (see Initial Testing Conditions below). This reduces cold-start bias and ensures that early information is matched across methods.

2) Iterative optimization

- **BASS-BO**: fit BASS surrogate on observed trials, optimize acquisition function, evaluate selected candidate, update surrogate, repeat.
- **GP-BO**: same loop with GP surrogate.
- **Random Search**: sample and evaluate candidates independently each iteration.
- **TPE**: the same ask-and-tell loop through Optuna's sampler, seeded with the identical initial design injected as completed trials.

3) Budget and stopping A fixed evaluation budget is used per task to make efficiency comparisons meaningful. Every method is run for the same number of iterations after the shared initial design, and best-so-far performance is recorded at each step.

4) Repetitions and randomization Each configuration is repeated across multiple random seeds. Because every method starts from the same per-seed initial design, the design is paired: each seed yields a directly comparable final result for every method, which is what the paired statistics below exploit.

6.2.4 Evaluation

As for evaluation of the methods, the central trade-off at the heart of Bayesian Optimization is that we want to achieve great returns at the lowest possible cost. On those axes, we define a performance metric and some efficiency metrics as well to test out our methods. The performance metric is simply the best objective value reached within budget. As for the efficiency metric, we track the evaluations over time to see how the methods behave under different constraints.

6.2.5 Statistical Analysis

Two complementary summaries are reported. First, convergence is shown as the mean best-so-far value per iteration with a 95% confidence interval across seeds, which conveys both speed and variability. Second, because all methods share the same initial design on each seed, final performance is compared with paired statistics: for each method we report the number of seeds in which it strictly beats, ties, or loses to the Random Search baseline, together with a paired Wilcoxon signed-rank test on the final best values. The pairing matters because final best-so-far values are minima and therefore skewed; a method can trail on the mean while winning most seeds, and the per-seed comparison is robust to exactly that.

6.2.6 Initial Testing Conditions

The initial testing conditions used in this study are listed below. Full implementation details, including scripts, targets, and output files, are available in the project repository: `github.com/AdrianTJ/BayesianOptim_BASS`.

- Branin-Hoo function ($2d$, bounded continuous domain)
- Rastrigin ($4d$, bounded continuous domain)
- Synthetic non-smooth surface ($3d$, unit cube; deliberately violates smoothness assumptions — see its subsection below)
- Func-2C (2 categorical + 2 continuous, mixed domain)
- Func-3C (3 categorical + 2 continuous, mixed domain)
- Cat-Ackley (purely categorical domain: d inputs with L levels each, run at three sizes — easy (d=3, L=5), medium (d=4, L=7), and hard (d=6, L=11); see the Cat-Ackley section below)

All methods are compared under a fixed evaluation budget of 80 iterations after the initial design, with an identical initialization strategy. For each replicate, an initial design of size $n_0 = \max(2d + 1, 8)$ is generated via maximin Latin Hypercube Sampling (LHS) where d is the dimension of the particular problem, and each optimizer proceeds for a fixed number of iterations. Results are aggregated over multiple random seeds (in our testing, 25 of them), and convergence is summarized using the best-so-far objective value at each iteration. Particular care was taken for these function evaluations to ensure reproducibility, so the seed always starts at the default value of 1001 and proceeds through to seed 1025. The seeds are independent across replicates and are fanned out across parallel workers; the surrogate fits and random draws are made worker-local so that results are identical regardless of how the work is distributed.

6.2.7 Acquisition and Candidate Generation

A deliberate design choice in this implementation is that BASS-BO and GP-BO are placed on equal footing, so that any measured difference can be attributed to the surrogate rather than to the surrounding machinery. Concretely, both methods share the same acquisition criterion and the same candidate generator, and neither exposes an exploration weight that has to be tuned.

The acquisition criterion is Expected Improvement (EI), which is parameter-free: it has no exploration constant to set, since the exploration–exploitation balance follows automatically from the surrogate's own predictive uncertainty. For GP-BO this is the usual closed-form EI under the Gaussian predictive. For BASS-BO, as described in the previous chapter, EI is computed by Monte Carlo over the retained posterior draws of the spline surface, which needs no Gaussian assumption. As a lighter alternative the implementation also supports Thompson sampling for the BASS surrogate, selected through the single `acquisition` setting (`"ei"` or `"thompson"`); all results reported here use Expected Improvement.

Because the acquisition is maximized over a finite pool rather than by continuous optimization, each iteration scores a fresh set of candidate points. The candidate generator is itself parameter-free, schema-aware, and shared by both surrogates. Half of the `n_cand` candidates are drawn from a space-filling Latin Hypercube over the whole domain (global exploration; categorical coordinates decode to uniformly distributed levels). The other half are local moves around the current best point, and "local" is defined per coordinate type. Continuous coordinates receive a Gaussian perturbation whose width is set to the incumbent's nearest-neighbour distance among the eval-

uated points — read off the data rather than tuned, and shrinking naturally as the design fills in around the optimum. Categorical coordinates instead make Hamming-local moves: each coordinate independently keeps the incumbent's level or, with probability $1/n_{\text{cat}}$ (one expected flip per candidate, where n_{cat} is the number of categorical inputs), switches to a uniformly random *other* level. A Gaussian nudge would be meaningless for an unordered factor, whereas a level flip is the correct notion of "nearby." Two details of this scheme matter in practice. On mixed problems a local candidate may flip *nothing*, keeping the incumbent's full categorical combination while refining only its continuous coordinates — exactly the local-exploitation move that lets a model-based method outrun random sampling once a good combination is found. On purely categorical problems, where a zero-flip candidate would merely duplicate the incumbent, at least one flip is always forced.

The remaining settings are experimental hygiene rather than tuning knobs: `n_cand`, the size of the candidate pool scored per iteration; `eps`, a small jitter added to the GP predictive variance for numerical stability; and `dup_tol`, the distance below which two points are treated as duplicates and excluded from re-evaluation. Duplicate detection operates on a canonicalized representation in which every categorical coordinate is snapped to a fixed representative of its decoded level, so two encodings that decode to the same level combination are recognized as the same input: on a deterministic objective, re-evaluating a known combination is a wasted iteration, and the canonical check prevents it for every method, Random Search included. Ties in the acquisition score — common in early iterations, when a nearly flat posterior makes Expected Improvement almost constant across the pool — are broken uniformly at random rather than by candidate order. Internally, the BASS surrogate is fit with a reduced RJMCMC budget relative to the package default, trading a little posterior resolution for speed inside the optimization loop; these are numerical settings of the sampler, not controls over the optimizer's behavior. The upshot is that the comparison in this chapter has essentially no free parameters on either side, which is precisely what makes a clean surrogate-versus-surrogate reading possible.

6.2.8 TPE Sensitivity to Its Gamma Hyperparameter

TPE is not parameter-free in the same sense. As discussed in the surrogate models chapter, Optuna's `TPESampler` reduces, for our purposes, to a single hyperparameter worth tuning: $\gamma \in [0, 1]$, the quantile of completed trials treated as "good" versus "bad." Everywhere else in this chapter TPE is run with Optuna's library default for γ, exactly as a practitioner reaching for the reference implementation would. To check whether that default is doing

meaningful work – and, by extension, whether BASS-BO's complete absence of such a knob is a genuine practical advantage rather than a cosmetic one – we ran a separate ablation: γ was swept over $\{0.10, 0.25, 0.50, 0.75\}$ (0.25 sits close to Optuna's own default behavior) on one continuous benchmark (Branin) and one purely categorical one (Cat-Ackley, at its hard size d=6, L=11), under the same budget, repetition count, and seeding protocol as the rest of this chapter. The resulting four TPE curves were plotted alongside the single, unsweepable BASS-BO and GP-BO reference curves, so the contrast is visual as much as numerical: where TPE's curves fan out across configurations, BASS-BO and GP-BO have nothing to fan out. The script and full protocol are in `code_files/2_tpe_sensitivity/`.

6.3 Results

What follows are the graphs representing some runs from the BASS-BO cycle, compared to GP-BO and Random Search. We ran these optimizers across the benchmark functions described above, spanning smooth continuous problems as well as categorical and mixed-variable ones, so as to get wider coverage into the actual performance of the optimizers across qualitatively different regimes.

6.3.1 Branin-Hoo Function

Formally, defined as:

$$f(x_1, x_2) = \left(x_2 - \frac{5.1}{4\pi^2}x_1^2 + \frac{5}{\pi}x_1 - 6\right)^2 + 10\left(1 - \frac{1}{8\pi}\right)\cos(x_1) + 10, \quad (6.1)$$

with $x_1 \in [-5, 10]$, $x_2 \in [0, 15]$.

Figure 6.2 shows the convergence results. GP-BO behaves exactly as its favorable assumptions predict: it reaches the global minimum ($f^* \approx 0.3979$) within roughly fifteen iterations on essentially every seed (mean final value 0.398, standard deviation 4×10^{-4}), and beats Random Search on all 25 paired seeds ($p = 1.3\times10^{-5}$). BASS-BO also clearly beats Random Search (23 wins, 2 losses across seeds, $p = 4.3 \times 10^{-5}$; mean final value 0.470), but converges more slowly and finishes behind GP-BO on every one of the 25 paired seeds. On this smooth, low-dimensional problem the Gaussian process is simply the better-matched surrogate, which is the expected baseline result rather than a surprise; the summary statistics for all continuous benchmarks are collected in Table 6.1.

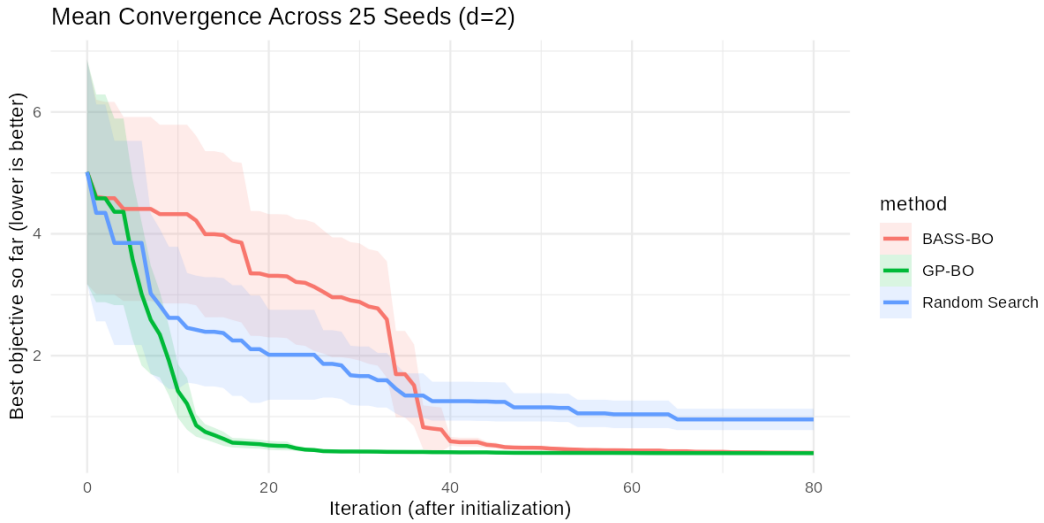

Figure 6.1: The Branin-Hoo surface on its physical domain. Its three global minima and smooth structure make it the classic sanity-check benchmark for continuous Bayesian optimization.

6.3.2 Rastrigin Function

$$f(\mathbf{x}) = An + \sum_{i=1}^{n} \left[x_i^2 - A\cos\left(2\pi x_i\right) \right], \tag{6.2}$$

$$\mathbf{x} = (x_1, \ldots, x_n), \qquad x_i \in [-5.12,\, 5.12], \quad i = 1, \ldots, n,$$

where $A = 10$ is the standard choice. We used the 4 dimensional version of this function.

Rastrigin is highly multimodal — a lattice of local minima superimposed on a quadratic bowl — and in four dimensions no method comes close to the global minimum within the 80-evaluation budget, so the comparison is about relative progress. As Figure 6.3 shows, GP-BO is again clearly the strongest method (mean final value 13.98), beating Random Search on 21 of 25 seeds ($p = 6.8 \times 10^{-5}$) and BASS-BO on 21 of 25 paired seeds ($p = 0.0012$). BASS-BO does retain a real but more modest edge over Random Search (mean 23.43 versus 28.80; 16 wins, 2 ties, 7 losses, $p = 0.018$). The pattern of the Branin result repeats at a harder difficulty: both surrogate-guided methods clear the Random Search floor, and the Gaussian process converts its smoothness prior into a substantial advantage even though the surface is multimodal, because it is still continuous and stationary — exactly the structure a GP kernel encodes.

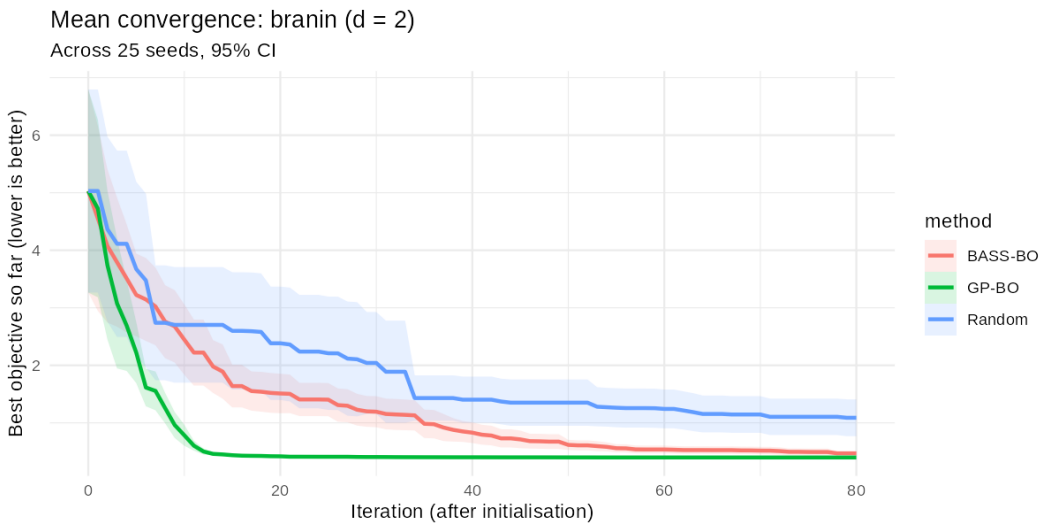

Figure 6.2: Branin-Hoo: mean best-so-far objective value per iteration across 25 seeds (1001–1025), budget 80 evaluations after a shared initial design; shaded bands are 95% confidence intervals across seeds.

6.3.3 Synthetic Non-Smooth Surface

The two benchmarks above are exactly the terrain a Gaussian process is built for, so we also include the hand-constructed synthetic surface used during the development of this pipeline, run here in three dimensions on the unit cube. It superimposes, on a global quadratic bowl, a decaying oscillation along the first axis, an oscillation in the second, a sharp Gaussian penalty bump and a Gaussian reward valley, and — most importantly — a jump discontinuity along the first axis. The jump is the deliberate provocation: a stationary GP kernel must smooth over it, while BASS's piecewise basis can in principle represent it. The full definition is in `code_files/R/objectives/synthetic.R` in the project repository.

The outcome (Figure 6.4) is a useful calibration of that intuition. Both surrogates clearly beat Random Search (BASS-BO: 20 wins, 5 losses, $p = 3.5 \times 10^{-4}$; GP-BO: 18 wins, 7 losses, $p = 0.0024$). Between the surrogates, however, the discontinuity buys BASS-BO parity rather than victory: GP-BO reaches its plateau within roughly twenty iterations while BASS-BO improves more gradually, and by the end of the budget their final values are statistically indistinguishable (mean -0.767 versus -0.755; paired Wilcoxon $p = 0.79$). This is the one continuous benchmark on which BASS-BO matches GP-BO, and it is also the one whose structure was designed to disadvantage the GP — an honest reading is that non-smoothness erodes the

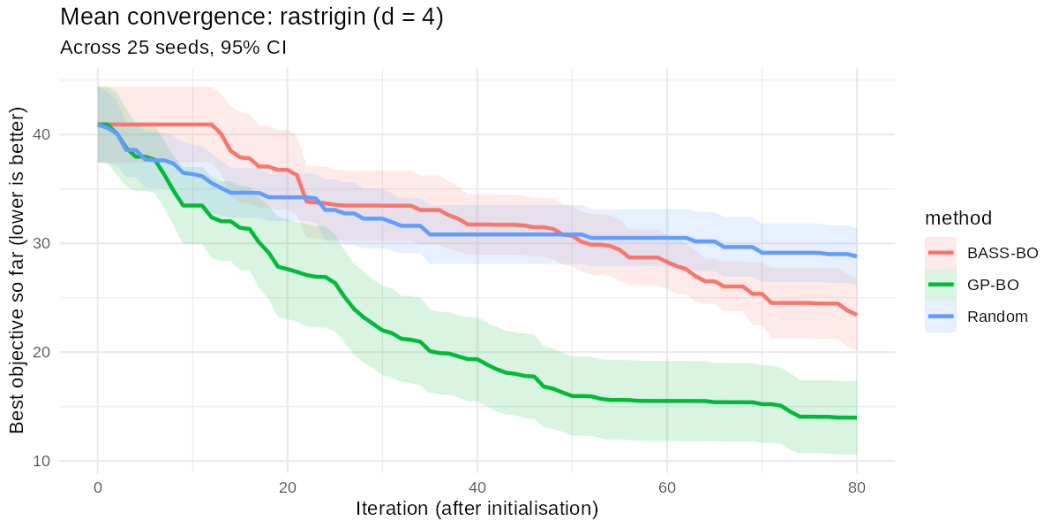

Figure 6.3: Rastrigin ($d = 4$): mean best-so-far objective value per iteration across 25 seeds (1001–1025), budget 80 evaluations after a shared initial design; shaded bands are 95% confidence intervals across seeds.

GP's edge without handing BASS one.

6.4 Categorical and Mixed-Variable Benchmarks

The benchmarks above are all smooth and continuous, the regime in which Gaussian-process surrogates are at their strongest. The purpose of this section is to test the capability that motivates the use of BASS in the first place, and which a standard Gaussian process does not have natively: optimization over categorical inputs. Recall from the previous chapters that the GP kernel measures similarity through a numeric distance between inputs, so an unordered categorical variable has no natural representation in it, whereas BASS models a categorical predictor directly, through basis functions defined on subsets of its levels.

To make the comparison concrete and fair, we treat the two surrogates exactly as their assumptions allow. BASS-BO is given the categorical inputs as genuine factors, so it uses its categorical basis. GP-BO is given the only thing a continuous-only surrogate can use, namely a continuous relaxation in which each categorical coordinate is a number in $[0, 1]$ that is decoded to a level. This relaxation is the baseline treatment of categorical inputs in GP-based optimization, formalized by [Garrido-Merchán and Hernández-Lobato,

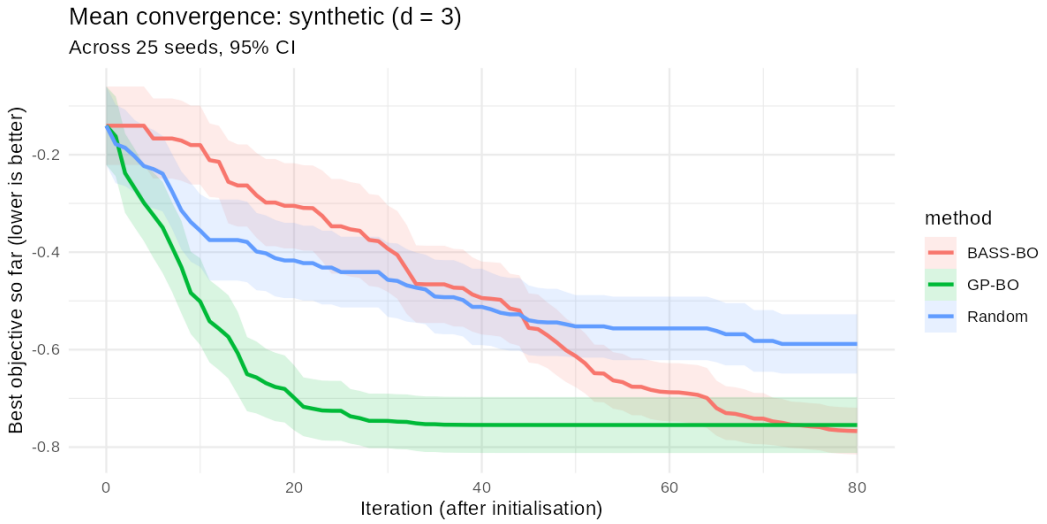

Figure 6.4: Synthetic non-smooth surface ($d = 3$): mean best-so-far objective value per iteration across 25 seeds (1001–1025), budget 80 evaluations after a shared initial design; shaded bands are 95% confidence intervals across seeds.

2020], who additionally transform the covariance function itself; our GP-BO stops at the plain relaxation, which is precisely the workaround a practitioner is forced into when applying an off-the-shelf GP to a categorical problem. Random Search, which samples levels uniformly, serves as the reference, and the TPE baseline serves as the harder test: it handles categorical inputs natively, so a strong showing against it speaks to BASS-BO's merits beyond simply not being a Gaussian process. We use three benchmarks, chosen to range from mixed continuous–categorical to purely categorical.

One clarification about what is, and is not, being claimed here. The categorical awareness in this pipeline lives in the shared *search machinery* — the schema-aware candidate generator and the combination-level duplicate handling described earlier — not in the GP surrogate, which remains categorically blind by design. That machinery follows established practice rather than contributing anything new: proposing candidates through a mix of global random samples and local moves around the incumbent, including one-exchange flips of categorical values, is how SMAC optimizes its acquisition over configuration spaces [Hutter et al., 2011], and Hamming-neighbourhood locality around the incumbent is the organizing idea of trust-region methods for categorical domains such as Casmopolitan [Wan et al., 2021b]. What the shared machinery buys this thesis is methodological, and it strengthens the baseline rather than the proposed method: GP-BO here is *stronger* than the naive relaxation a practitioner would assemble on their own, because it ben-

Benchmark	Method	Final best (mean ± sd)	W/T/L	p
Branin ($2d$)	GP-BO	0.398 ± 0.0004	25/0/0	1.3×10^{-5}
	BASS-BO	0.470 ± 0.088	23/0/2	4.3×10^{-5}
	Random	1.088 ± 0.815	—	—
Rastrigin ($4d$)	GP-BO	13.98 ± 8.62	21/0/4	6.8×10^{-5}
	BASS-BO	23.43 ± 8.33	16/2/7	0.018
	Random	28.80 ± 6.61	—	—
Synthetic ($3d$)	BASS-BO	-0.767 ± 0.122	20/0/5	3.5×10^{-4}
	GP-BO	-0.755 ± 0.146	18/0/7	0.0024
	Random	-0.588 ± 0.155	—	—

Table 6.1: Continuous benchmarks: final best objective value after 80 evaluations (mean ± standard deviation across 25 seeds, 1001–1025), per-seed wins/ties/losses against the paired Random Search baseline, and the two-sided paired Wilcoxon signed-rank p-value. Methods are ordered by mean final value within each benchmark.

efits from the same categorical-aware proposals and duplicate handling that BASS-BO does. Any advantage BASS-BO shows in what follows is therefore attributable to the surrogate itself, not to a handicapped opponent.

6.4.1 Func-2C and Func-3C (mixed)

Func-2C and Func-3C are standard mixed-variable benchmarks introduced with the CoCaBO method [Ru et al., 2020]. In both, the categorical variables select which of three classic two-dimensional functions, Rosenbrock, Six-Hump Camel, and Beale, are combined, and two continuous variables $x_1, x_2 \in [-1, 1]$ supply their arguments (rescaled internally to $[-2, 2]$). Func-2C has two categorical variables with 3 and 5 levels respectively (so 15 categorical combinations); Func-3C adds a third categorical variable with 4 levels (for 60 combinations) whose levels contribute additional, weighted copies of the component functions. Writing $h_1, h_2, (h_3)$ for the categorical choices and $f_{\mathrm{R}}, f_{\mathrm{C}}, f_{\mathrm{B}}$ for the (scaled) Rosenbrock, Six-Hump Camel and Beale functions, Func-2C is

$$f(h_1, h_2, x_1, x_2) = f^{(h_1)}(x_1, x_2) + f^{(h_2)}(x_1, x_2),$$

with h_1 selecting among $\{f_{\mathrm{R}}, f_{\mathrm{C}}, f_{\mathrm{B}}\}$ and h_2 selecting Rosenbrock, Six-Hump Camel, or Beale for its remaining levels; Func-3C adds a third term whose component and weight depend on h_3. Both are minimized. The reason these benchmarks are informative is that the relationship between a categorical level and the resulting surface is not continuous in the level index, so a relaxation that treats the level as an ordered number is modelling a discontinuous

target with a smooth prior.

The results (Figures 6.5 and 6.6, Table 6.2) are the clearest negative finding of this thesis, and we report them as such. On both mixed benchmarks BASS-BO is not merely behind the other surrogate-guided methods — it fails to clear the Random Search floor. On Func-2C it beats Random Search on only 5 of 25 paired seeds while losing on 18 ($p = 0.043$); on Func-3C the count is 8 wins against 14 losses ($p = 0.048$). In both cases the paired test is significant at the 5% level *in Random Search's favor*. Meanwhile GP-BO, running on the supposedly disadvantageous continuous relaxation, is the best method on both problems (20/1/4 and 23/0/2 against Random Search), with TPE second; GP-BO beats BASS-BO on 23 and 25 of the 25 paired seeds respectively.

Two aspects of the experimental design sharpen what this result means. First, because the search machinery — candidate generation, duplicate handling, initialization — is shared across methods, and because the oracle diagnostics described in the repository confirmed that this machinery permits reaching the optimum of these benchmarks, the failure is attributable to the BASS surrogate-acquisition combination itself, not to the surrounding loop. Second, held-out diagnostics of the BASS fit on these functions show moderate predictive correlation at these sample sizes, so the surrogate is not simply unable to model the surface. A plausible but unproven explanation is that the Monte Carlo Expected Improvement over a modest number of retained posterior draws is too coarse to resolve the fine continuous refinement these composite surfaces reward once a good categorical combination is found; we flag this as a hypothesis for future work rather than a conclusion. What can be stated from the data is the outcome: on mixed continuous–categorical spaces of this type, at this budget, BASS-BO as implemented here is not competitive.

6.4.2 Cat-Ackley (purely categorical)

To stress the categorical capability in isolation, we also use a purely categorical encoding of the Ackley function [Surjanovic and Bingham, 2024], following the way combinatorial Bayesian optimization benchmarks are commonly constructed from continuous test functions [Oh et al., 2019]. Each of the d inputs is a categorical variable with L levels, and level k of input j is mapped to a grid value through a *fixed random permutation* π_j, that is, the decoded coordinate is $g[\pi_j(k)]$ where g is an evenly spaced grid on Ackley's domain that includes 0. The objective is then the Ackley value at the decoded point, minimized, with a known global minimum of 0 attained at the level that each permutation maps to the grid value 0. The permutation is the

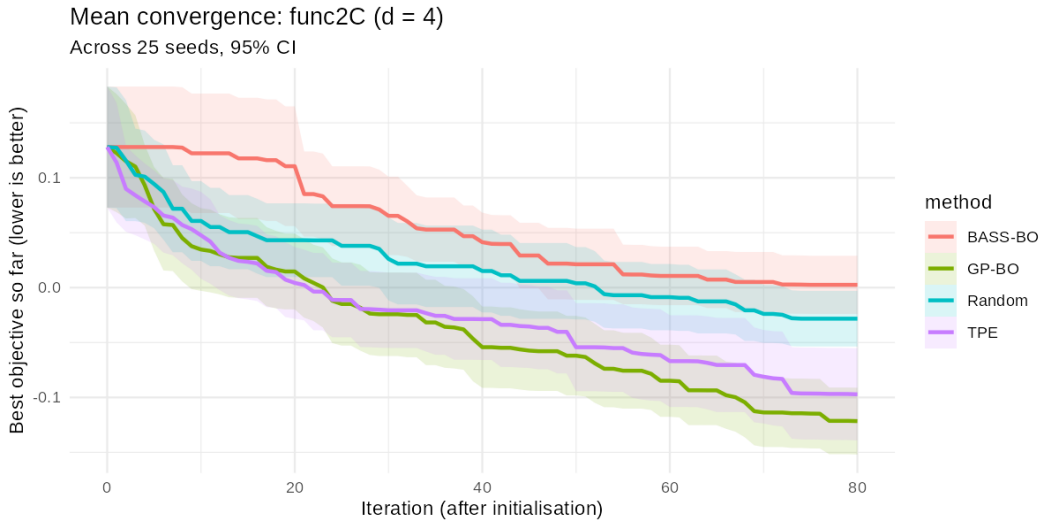

Figure 6.5: Func-2C (2 categorical + 2 continuous): mean best-so-far objective value per iteration across 25 seeds (1001–1025), budget 80 evaluations after a shared initial design; shaded bands are 95% confidence intervals across seeds. BASS-BO finishes above the Random Search baseline.

crucial ingredient: it destroys any monotone relationship between a level's index and its effect, so the continuous relaxation that GP-BO must use sees a jagged, meaningless surface, while BASS, splitting on subsets of levels, can still isolate the good ones. This is the cleanest illustration of the structural difference between the two surrogates.

The size of the search space, L^d combinations, is itself a protocol choice, and we run the benchmark at three sizes rather than one. The easy instance (d=3, L=5: 125 combinations) is small enough that the 80-evaluation budget can meaningfully cover it, so it tests whether a method can actually *solve* a categorical problem when solving is feasible — a capability demonstration. The medium instance (d=4, L=7: 2,401 combinations) and the hard instance (d=6, L=11: roughly 1.77 million combinations) progressively outgrow the budget; on the hard instance no method can be expected to reach the optimum in 80 evaluations, and the honest question becomes relative progress — how much better than uniform sampling a surrogate's guidance is when it can only ever see a vanishing fraction of the space. Reporting all three sizes separates what a method *can do* from what any method *could do* at this budget, and keeps single-instance results from overgeneralizing in either direction.

Figures 6.7–6.9 and Table 6.3 report the three instances, and here the picture reverses relative to the mixed benchmarks. On the easy instance the

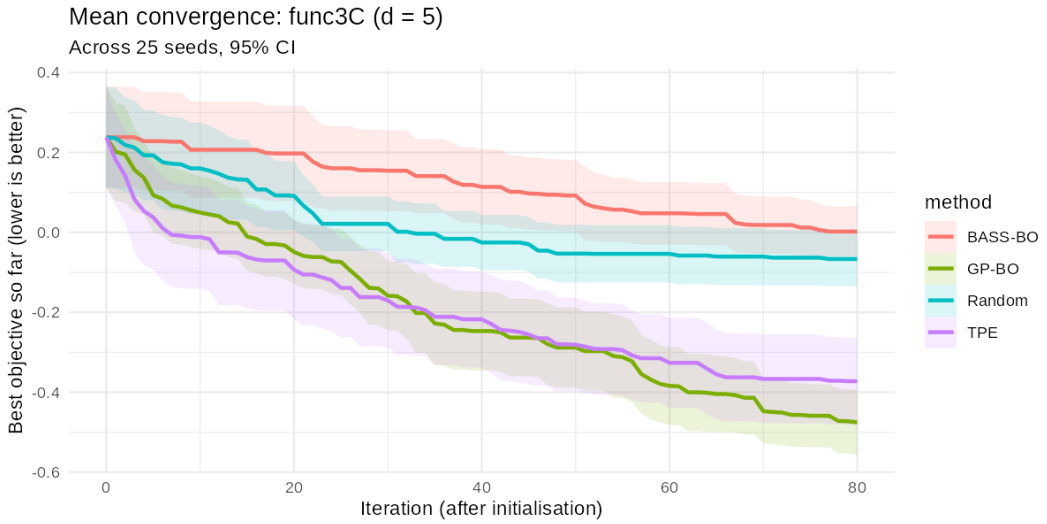

Figure 6.6: Func-3C (3 categorical + 2 continuous): mean best-so-far objective value per iteration across 25 seeds (1001–1025), budget 80 evaluations after a shared initial design; shaded bands are 95% confidence intervals across seeds.

capability question is answered affirmatively and decisively: BASS-BO finds the exact global optimum on all 25 seeds, in a median of only 4 iterations after the initial design. It is not alone — GP-BO also solves all 25 seeds (median 6 iterations), TPE solves 20, and even Random Search stumbles onto the optimum in 17 of 25 seeds, which is why the paired win counts (8/17/0 for both BO methods) are dominated by ties: on a 125-combination space, an 80-evaluation budget with duplicate avoidance makes the problem genuinely solvable for everyone, and the meaningful difference is *speed*, visible in Figure 6.7.

The medium instance (2,401 combinations, so the budget covers under 4% of the space) is where the methods separate most cleanly, and it is BASS-BO's strongest result in this thesis. BASS-BO still solves the problem exactly on 19 of 25 seeds (median 36 iterations); GP-BO solves 15 (median 50), TPE 7, and Random Search 1. In paired terms BASS-BO beats Random Search 23/1/1 ($p = 4.0 \times 10^{-5}$) and never loses to TPE on any seed (14 wins, 11 ties). Its edge over GP-BO points the right way (10 wins, 11 ties, 4 losses across seeds) but does not reach significance ($p = 0.097$).

On the hard instance ($\sim$1.77 million combinations) no method solves the problem, as intended, and the question is relative progress. Both BO methods make substantial headway — BASS-BO beats Random Search on 25 of 25 paired seeds ($p = 1.3 \times 10^{-5}$), GP-BO on 24 — and both finish well ahead of

Benchmark	Method	Final best (mean ± sd)	W/T/L	p
Func-2C	GP-BO	-0.122 ± 0.077	20/1/4	5.2×10^{-4}
	TPE	-0.097 ± 0.107	18/1/6	0.0053
	Random	-0.028 ± 0.065	—	—
	BASS-BO	0.003 ± 0.067	5/2/18	0.043
Func-3C	GP-BO	-0.475 ± 0.205	23/0/2	5.4×10^{-5}
	TPE	-0.372 ± 0.279	19/1/5	7.1×10^{-4}
	Random	-0.067 ± 0.173	—	—
	BASS-BO	0.002 ± 0.161	8/3/14	0.048

Table 6.2: Mixed benchmarks: final best objective value after 80 evaluations (mean ± standard deviation across 25 seeds, 1001–1025), per-seed wins/ties/losses against the paired Random Search baseline, and the two-sided paired Wilcoxon signed-rank p-value. Methods are ordered by mean final value; note that BASS-BO ranks below Random Search on both benchmarks, and its significant p-values correspond to *losses*.

TPE (BASS-BO beats TPE on 23 of 25 seeds with no losses). Between the two surrogates there is nothing to choose: mean final values of 12.14 (BASS-BO) and 11.99 (GP-BO), paired $p = 0.67$. The three sizes read together say that BASS-BO's categorical machinery works, solves what is solvable, degrades gracefully, and consistently outperforms both baselines that lack a regression surrogate — but that a GP on the continuous relaxation, given the same categorical-aware search machinery, is essentially its equal on two of the three instances. We return to this last observation in the interpretation section, because it says as much about the machinery as about the surrogates.

6.4.3 TPE Hyperparameter Sensitivity

As described in Section 6.2.8, we swept TPE's γ hyperparameter over $\{0.10, 0.25, 0.50, 0.75\}$ on Branin and Cat-Ackley, holding everything else (budget, repetitions, seeding, initial design) fixed to the same protocol used throughout this chapter. The figures below plot the four resulting TPE curves alongside the single BASS-BO and GP-BO reference curves.

The knob turns out to do a great deal of work (Figures 6.10 and 6.11). On Branin, TPE's mean final value ranges from 0.494 at $\gamma = 0.10$ to 4.55 at $\gamma = 0.75$ — a spread of 4.06, nearly an order of magnitude between the best and worst configurations, with the two aggressive settings ($\gamma \in \{0.50, 0.75\}$) finishing *behind* Random Search. On the hard Cat-Ackley instance the spread is 4.12 (best 15.67 at $\gamma = 0.25$, worst 19.80 at $\gamma = 0.75$), and again the two aggressive settings do no better than Random Search (18.65); moreover, even

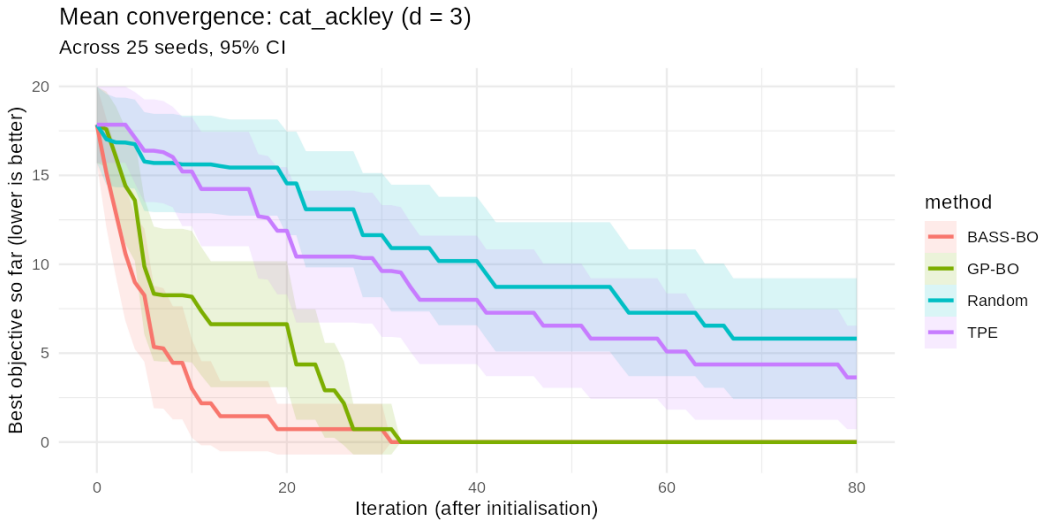

Figure 6.7: Cat-Ackley, easy instance ($d = 3$, $L = 5$: 125 combinations): mean best-so-far objective value per iteration across 25 seeds (1001–1025), budget 80 evaluations after a shared initial design; shaded bands are 95% confidence intervals across seeds. Both BASS-BO and GP-BO reach the exact optimum on every seed.

the best-tuned TPE configuration remains well behind the single, knob-free BASS-BO (12.14) and GP-BO (11.99) curves. Two things follow. First, a practitioner using TPE inherits a hyperparameter whose poor settings cost more than the difference between entire methods, while BASS-BO and GP-BO under the parameter-free Expected Improvement acquisition simply have no analogous quantity to mis-set — the practical advantage this ablation was designed to probe is real. Second, the caveat cuts both ways: TPE at a well-chosen γ is respectable on Branin (0.494, close to BASS-BO's 0.470), so the argument is about the *risk* and *tuning burden* of the knob, not about TPE's ceiling.

6.5 Result Interpretation and Limitations

On the continuous benchmarks the pre-registered expectation held, and held clearly. Where the objective is smooth and stationary, GP-BO is the better optimizer by every measure: it beats BASS-BO on 25 of 25 paired seeds on Branin and 21 of 25 on Rastrigin, and it converges markedly faster. BASS-BO is a genuine improvement over Random Search on all three continuous problems, but it is a second-place method in the GP's home terrain, and

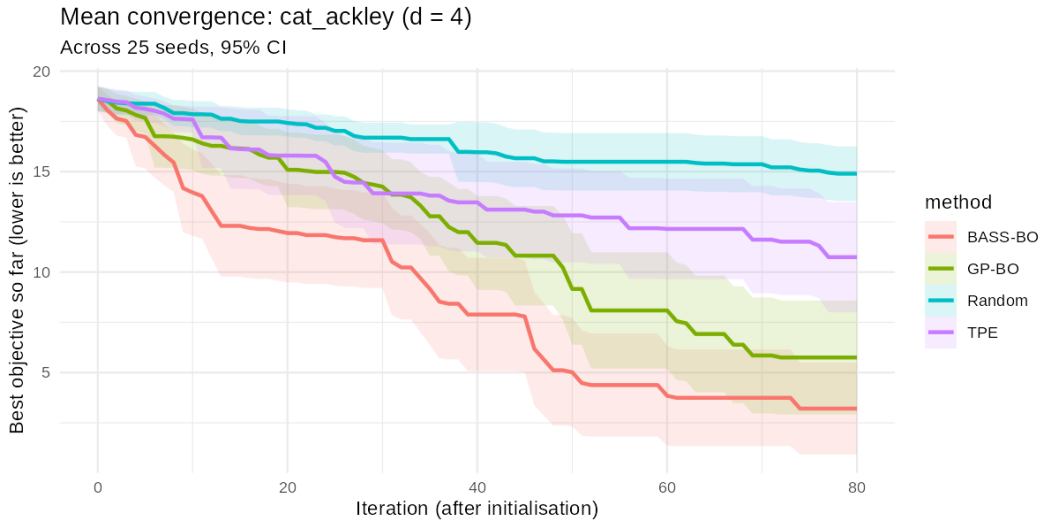

Figure 6.8: Cat-Ackley, medium instance ($d = 4$, $L = 7$: 2,401 combinations): mean best-so-far objective value per iteration across 25 seeds (1001–1025), budget 80 evaluations after a shared initial design; shaded bands are 95% confidence intervals across seeds.

nothing in these results argues for preferring it there. The one nuance comes from the non-smooth synthetic surface, whose jump discontinuity was constructed to violate the GP's assumptions: there the gap closes to statistical parity ($p = 0.79$), with GP-BO still faster early. Non-smoothness of this kind erodes the GP's advantage; on this evidence it does not create a BASS advantage.

On the purely categorical benchmarks the capability that motivated this thesis is demonstrated, with an important qualification. Read together, the three Cat-Ackley sizes show BASS-BO solving what is solvable (25 of 25 seeds on the easy instance, in a median of 4 iterations), holding its edge as the space outgrows the budget (its strongest separation on the medium instance, where it solves 19 of 25 seeds against GP-BO's 15, TPE's 7, and Random Search's 1), and degrading gracefully on the hard instance, where it beats Random Search on every one of the 25 paired seeds and TPE on 23 of them while never losing to either. Against the baselines that lack a regression surrogate, the categorical story is unambiguous. The qualification concerns GP-BO: given the same categorical-aware search machinery, the GP on a plain continuous relaxation matches BASS-BO on the easy and hard instances and trails without statistical significance on the medium one ($p = 0.097$). The structural argument — that a GP kernel cannot represent unordered levels — did not translate into the performance gap it suggests.

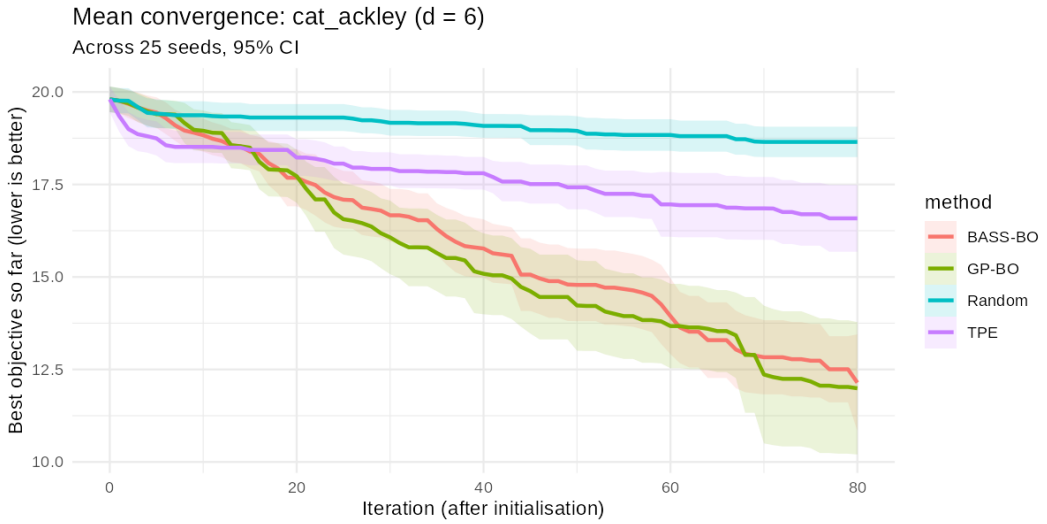

Figure 6.9: Cat-Ackley, hard instance ($d = 6$, $L = 11$: $\sim$1.77M combinations): mean best-so-far objective value per iteration across 25 seeds (1001–1025), budget 80 evaluations after a shared initial design; shaded bands are 95% confidence intervals across seeds. No method reaches the optimum; the comparison is relative progress.

We read this as evidence for the machinery-confound point developed in the conclusions: much of what looks like "categorical capability" in a Bayesian optimizer lives in the candidate generation and duplicate handling, and once those are shared and categorical-aware, the choice of surrogate matters less than the literature's method-to-method comparisons imply.

On the mixed benchmarks the result is negative and we state it without hedging: BASS-BO failed. It lost to Random Search under the paired test on both Func-2C (5 wins, 18 losses, $p = 0.043$) and Func-3C (8 wins, 14 losses, $p = 0.048$), while GP-BO and TPE both beat Random Search comfortably on the same seeds. The shared-machinery design is what makes this diagnosis informative rather than merely embarrassing: the search loop demonstrably permits solving these problems (GP-BO does so with the identical loop), so the failure localizes to the BASS surrogate-acquisition combination on mixed spaces at this budget. Whether the cause is the Monte Carlo resolution of the Expected Improvement over BASS's posterior draws, or a mismatch between the spline basis and these composite surfaces, is a question these experiments were not designed to answer; the honest summary is that BASS-BO, as implemented here, should not currently be recommended for mixed continuous–categorical problems, and that the failure mode was invisible on every purely categorical benchmark.

Instance	Method	Final best (mean ± sd)	W/T/L	p
Easy (d=3, L=5)	BASS-BO	0.00 ± 0.00	8/17/0	0.0060
	GP-BO	0.00 ± 0.00	8/17/0	0.0060
	TPE	3.64 ± 7.42	8/12/5	0.43
	Random	5.82 ± 8.66	—	—
Medium (d=4, L=7)	BASS-BO	3.21 ± 5.83	23/1/1	4.0×10^{-5}
	GP-BO	5.75 ± 7.23	22/3/0	4.2×10^{-5}
	TPE	10.74 ± 6.98	17/1/7	0.017
	Random	14.90 ± 3.45	—	—
Hard (d=6, L=11)	GP-BO	11.99 ± 4.56	24/0/1	1.5×10^{-5}
	BASS-BO	12.14 ± 3.33	25/0/0	1.3×10^{-5}
	TPE	16.58 ± 2.30	22/0/3	9.6×10^{-5}
	Random	18.65 ± 1.04	—	—

Table 6.3: Cat-Ackley at three sizes: final best objective value after 80 evaluations (mean ± standard deviation across 25 seeds, 1001–1025), per-seed wins/ties/losses against the paired Random Search baseline, and the two-sided paired Wilcoxon signed-rank p-value. On the easy instance a final value of 0.00 means the exact global optimum (to machine precision) on every seed; the many ties against Random Search occur because Random Search itself solves that instance on 17 of 25 seeds.

Four scope limits bound all of these conclusions. First, the benchmark suite is synthetic and low-dimensional (two to six input dimensions); no real hyperparameter-tuning task is reported, and behavior on noisy, higher-dimensional, or structured real objectives is untested here. Second, every result is at a single evaluation budget of 80 iterations; rankings can and do change with budget (BASS-BO's late catch-up on the synthetic surface is a case in point), and no claim is made about other budgets. Third, all objectives are deterministic, so the duplicate-avoidance logic and the paired statistics both exploit an exactness that noisy objectives would not provide. Fourth, the comparison covers one acquisition function (Expected Improvement) and one TPE implementation at its library default γ outside the dedicated ablation; different acquisition choices could shift the surrogate comparison in either direction. These limits are the natural agenda for the future-work chapter, and none of them soften the two headline findings: the categorical capability is real, and the mixed-variable failure is real.

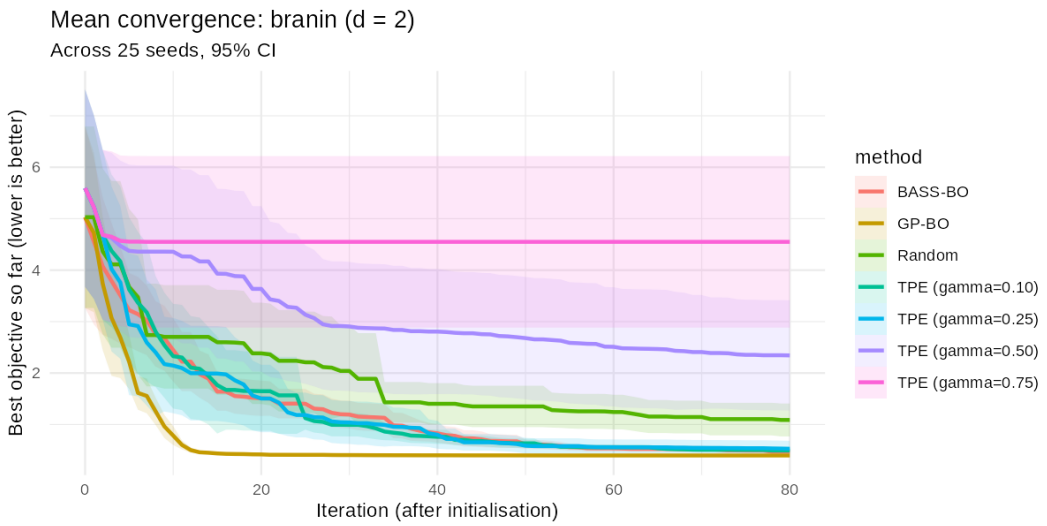

Figure 6.10: TPE γ-sensitivity on Branin: mean best-so-far objective value per iteration for TPE at $\gamma \in \{0.10, 0.25, 0.50, 0.75\}$, alongside the single BASS-BO and GP-BO reference curves and Random Search. 25 seeds (1001–1025), budget 80 evaluations after a shared initial design; shaded bands are 95% confidence intervals across seeds.

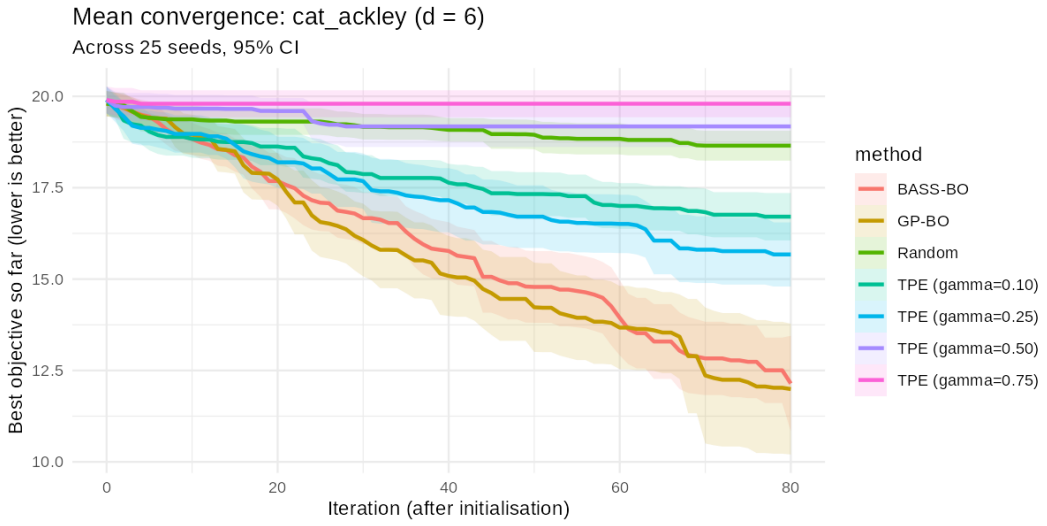

Figure 6.11: TPE γ-sensitivity on the hard Cat-Ackley instance ($d = 6$, $L = 11$): mean best-so-far objective value per iteration for TPE at $\gamma \in \{0.10, 0.25, 0.50, 0.75\}$, alongside the single BASS-BO and GP-BO reference curves and Random Search. 25 seeds (1001–1025), budget 80 evaluations after a shared initial design; shaded bands are 95% confidence intervals across seeds.

Chapter 7

Conclusions and Future Work

7.1 Conclusions

This thesis studied the use of Bayesian Adaptive Spline Surfaces (BASS) as surrogate models within a Bayesian Optimization framework for black-box optimization and hyperparameter tuning. The central objective was to determine whether a BASS-based optimizer can offer competitive sample efficiency when compared with established baselines, while also providing practical flexibility for difficult objective landscapes. To evaluate this, a complete experimental pipeline was developed in R, including multi-dimensional optimization support, reproducible seed-based replication, parallel execution, and standardized convergence and paired-statistics reporting against GP-BO, Random Search, and TPE.

The experimental chapter reports, for every benchmark, mean convergence curves with confidence intervals and per-seed paired comparisons against the Random Search baseline. On the smooth, low-dimensional continuous benchmarks, where GP surrogates are particularly well matched to the objective, the expected outcome held and held clearly: GP-BO converged fastest, reaching the Branin optimum on essentially every seed and beating BASS-BO on 25 of 25 paired seeds there and on 21 of 25 on Rastrigin. Both surrogate-guided methods cleared the Random Search floor on every continuous benchmark, and on the deliberately non-smooth synthetic surface — the one continuous problem built to violate the GP's assumptions — the two surrogates finished statistically tied. The paired win/tie/loss counts make every one of these comparisons explicit seed by seed.

A key conclusion of this work is that BASS-BO should not be interpreted as a universally superior replacement for GP-BO. Instead, the two should be seen as suited to different regimes. On the smooth,

low-dimensional continuous benchmarks, GP-BO is the natural favorite, both because its smoothness assumptions match these problems and because its closed-form predictive uncertainty is ideally matched to them; measured under the shared protocol, that advantage was decisive rather than marginal. On purely categorical problems the two surrogates proved nearly equal once both were given the same categorical-aware search machinery, and on the mixed benchmarks GP-BO was clearly superior — BASS-BO's advantage over the GP turned out to be narrower than the structural argument suggests. Importantly, in this implementation the two methods are placed on equal footing: they share the same parameter-free Expected Improvement acquisition and the same candidate generator, and BASS supplies its predictive uncertainty natively through its posterior draws rather than through any hand-tuned exploration schedule. The difference in performance is therefore attributable to the surrogate itself rather than to tuning, which is exactly what makes the comparison clean.

From a methodological perspective, one contribution of this thesis is the construction of a reproducible comparison in which the surrogate is the only moving part. Holding the full experimental protocol constant, including the initialization strategy, candidate pool size, duplicate handling, seed replication, and stopping budget, is what allows measured differences to be read as differences in model quality rather than in procedure. Without this consistency, apparent performance gaps can reflect the surrounding machinery rather than the surrogate.

The development of this pipeline produced direct evidence for how large that machinery effect can be, and we consider this a finding in its own right. In controlled oracle experiments — replacing the acquisition with the true objective, so that the candidate pool is the only binding constraint — a single detail of the candidate generator (whether local proposals may keep the incumbent's categorical combination while refining its continuous coordinates) determined whether *any* surrogate, including a perfect one, could approach the optimum of the mixed benchmarks: the restricted generator was beaten by the permissive one in every paired seed, and with the restricted generator even a perfect surrogate exploited no faster than the pool allowed. Likewise, before duplicate handling operated at the level of decoded categorical combinations, roughly two thirds of the evaluation budget on a small categorical benchmark was silently spent re-evaluating known combinations. Both effects are invisible in a standard convergence plot, and both would have been attributed to the surrogate had the machinery not been shared. The implication for the field is uncomfortable but useful: surrogate comparisons in mixed and categorical spaces are confounded by acquisition-optimization machinery unless that machinery is shared, categorical-aware, and audited

— and much of the published literature compares surrogates atop method-specific machinery. A systematic treatment of this confound, including the oracle-ceiling audit as a general diagnostic, is beyond the scope of this thesis and is being developed as a separate article.

The current study also has scope limits. The benchmark set spans two to six input dimensions and a single evaluation budget, and the continuous benchmarks in particular emphasize smooth functions where GP assumptions are favorable. The categorical and mixed-variable benchmarks probe the opposite regime; the Cat-Ackley family is deliberately run at three sizes so that capability (on a space the budget can cover) and scaling behavior (on spaces that outgrow it) are reported separately rather than conflated. Within that scope the categorical outcomes are unambiguous: BASS-BO solved the easy instance on all 25 seeds (as did GP-BO; TPE on 20, Random Search on 17), separated most clearly on the medium instance (solving 19 of 25 seeds, against 15 for GP-BO, 7 for TPE, and 1 for Random Search), and beat Random Search on all 25 paired seeds of the hard instance while never losing a paired seed to TPE at any size. The mixed benchmarks are the other side of that ledger: on Func-2C and Func-3C, BASS-BO lost to Random Search under the paired test (5 wins to 18 losses and 8 wins to 14 losses respectively, both significant at the 5% level), while GP-BO and TPE beat Random Search comfortably on the same seeds — a genuine failure mode of the method as implemented, reported as such in the experimental chapter. **Broader coverage, including more irregular, nonstationary, or interaction-heavy objectives**, remains an open empirical direction that the established codebase is now prepared to test.

The practical takeaway is therefore one of complementary strengths. For smooth, low-dimensional objectives under tight evaluation budgets, GP-BO is an excellent default, and the measured gap is not close: under the shared protocol it beat BASS-BO on every paired Branin seed and on 21 of 25 Rastrigin seeds, converging faster throughout. On such problems BASS-BO is a competent second choice that still clearly improves on Random Search, nothing more. **The distinguishing advantage of BASS-BO is structural rather than a matter of tuning: it accepts categorical variables natively**, through basis functions defined on subsets of their levels, and can therefore operate on mixed and purely categorical search spaces on which a standard Gaussian process must fall back on encodings or specialized kernels. This is the capability that the categorical benchmarks of the previous chapter are designed to exercise, and on purely categorical spaces the results bear it out; on mixed spaces they do not, and the recommendation that follows is correspondingly regime-specific. It is worth being clear that BASS is not the only surrogate with this property: tree- and density-based methods, the

Tree-structured Parzen Estimator chief among them, also handle categorical inputs natively, which is precisely why we benchmark against it rather than against a Gaussian process alone. The contribution is therefore not that BASS is uniquely categorical-capable, but that it brings a flexible, fully Bayesian spline surrogate to a setting the Gaussian process it is usually compared against cannot reach natively. On the purely categorical benchmarks it is more than competitive with that purpose-built categorical optimizer — BASS-BO never lost a paired seed to TPE on any Cat-Ackley instance, and beat it outright on 23 of 25 seeds on the hardest one. On the mixed benchmarks the order reverses: TPE and GP-BO clearly outperformed BASS-BO there, so the honest positioning is that BASS-BO is a strong choice for purely categorical search spaces and, in its present form, not a recommendable one for mixed continuous–categorical problems.

7.2 Future Work

Future work should extend this analysis in two directions. First, the categorical and mixed-variable evaluation should be deepened: scaling the number of categories and categorical dimensions further, enriching the categorical proposal distribution beyond the current per-coordinate Hamming moves (for example, correlated multi-coordinate moves informed by the surrogate's own level effects), and moving from synthetic benchmarks to real mixed hyperparameter-tuning tasks, where categorical choices such as model family or kernel sit alongside continuous parameters. Second, the continuous benchmark suite should be broadened to include more multimodal, higher-dimensional, and noisy objectives, so that conclusions are less tied to any one function class. Together these would sharpen the characterization of when BASS-BO is preferable to GP-BO.

In summary, this thesis delivers a reproducible and extensible framework for comparing surrogate-based Bayesian Optimization methods, and demonstrates that BASS can be integrated effectively into a multi-dimensional BO loop. The measured outcomes draw a sharper regime map than the structural argument alone would: on smooth continuous problems GP-BO dominates and BASS-BO is a clear-but-second improvement over Random Search; on purely categorical problems BASS-BO solves what the budget makes solvable and consistently outperforms both Random Search and TPE, with a GP on a shared-machinery continuous relaxation as its only equal; and on mixed continuous–categorical problems BASS-BO fails to beat Random Search, a negative result this thesis reports with the same prominence as the positive ones. The main outcome is therefore both empirical and methodological: em-

pirically, BASS-BO earns a place as a surrogate for purely categorical search spaces; methodologically, the shared, categorical-aware, audited search machinery is what made both the positive and the negative findings attributable to the surrogate itself — and it is also what revealed how much of the apparent difference between "categorical" and "non-categorical" optimizers lives in that machinery rather than in the models.

Bibliography

Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 2623–2631, 2019.

Maximilian Balandat, Brian Karrer, Daniel Jiang, Samuel Daulton, Ben Letham, Andrew G Wilson, and Eytan Bakshy. Botorch: A framework for efficient monte-carlo bayesian optimization. *Advances in neural information processing systems*, 33:21524–21538, 2020.

James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of machine learning research*, 13(2), 2012.

James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. *Advances in neural information processing systems*, 24, 2011.

James Bergstra, Dan Yamins, David D Cox, et al. Hyperopt: A python library for optimizing the hyperparameters of machine learning algorithms. In *Proceedings of the 12th Python in science conference*, volume 13, page 20, 2013.

P Billingsley. *Probability and Measure*. Wiley Series in Probability and Statistics. Wiley, 2012. ISBN 9781118341919.

Leo Breiman. *Classification and regression trees*. Routledge, 2017.

Kai Lai Chung. *Markov chains with stationary transition probabilities*. Springer, Berlin, 1960. ISBN 978-3-642-49686-8.

David G T Denison, Bani K Mallick, and Adrian F M Smith. Bayesian mars. *Statistics and Computing*, 8:337–346, 1998.

Devin Francom and Bruno Sansó. Bass: An r package for fitting and performing sensitivity analysis of bayesian adaptive spline surfaces. *Journal of Statistical Software*, 94(8), 2020.

Jerome H Friedman. Multivariate adaptive regression splines. *The Annals of Statistics*, 19(1), Mar 1991a. ISSN 0090-5364. doi: 10.1214/aos/1176347963. URL https://projecteuclid.org/journals/annals-of-statistics/volume-19/issue-1/Multivariate-Adaptive-Regression-Splines/10.1214/aos/1176347963.full.

Jerome H Friedman. Multivariate adaptive regression splines. *The annals of statistics*, 19(1):1–67, 1991b.

Roman Garnett. *Bayesian Optimization*. Cambridge University Press, 2023. to appear.

Eduardo C Garrido-Merchán and Daniel Hernández-Lobato. Dealing with categorical and integer-valued variables in bayesian optimization with gaussian processes. *Neurocomputing*, 380:20–35, 2020.

Eduardo C Garrido-Merchán and Daniel Hernández-Lobato. Dealing with categorical and integer-valued variables in bayesian optimization with gaussian processes. *Neurocomputing*, 380:20–35, Mar 2020. ISSN 09252312. doi: 10.1016/j.neucom.2019.11.004.

Peter J Green. Reversible jump markov chain monte carlo computation and bayesian model determination. *Biometrika*, 82(4):711–732, 1995.

Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? *Advances in Neural Information Processing Systems*, 35:507–520, 2022.

Johannes M Hohendorff and J Rosenthal. An introduction to markov chain monte carlo. *University of Toronto, Department of Statistics*, 2005.

Frank Hutter, Holger H Hoos, and Kevin Leyton-Brown. Sequential model-based optimization for general algorithm configuration. In *International Conference on Learning and Intelligent Optimization (LION 5)*, pages 507–523. Springer, 2011.

Emilie Kaufmann, Olivier Cappé, and Aurélien Garivier. On bayesian upper confidence bounds for bandit problems. In *Artificial intelligence and statistics*, pages 592–600. PMLR, 2012a.

Emilie Kaufmann, Nathaniel Korda, and Rémi Munos. Thompson sampling: An asymptotically optimal finite-time analysis. In *International conference on algorithmic learning theory*, pages 199–213. Springer, 2012b.

Will Koehrsen. A conceptual explanation of bayesian hyperparameter optimization for machine learning, 2018. URL https://towardsdatascience.com/a-conceptual-explanation-of-bayesian-model-based-hyperparameter-optimization-for- Accessed: 15 Feb 2023.

Michael D McKay, Richard J Beckman, and William J Conover. A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. *Technometrics*, 42(1):55–61, 2000.

Changyong Oh, Jakub M Tomczak, Efstratios Gavves, and Max Welling. Combinatorial bayesian optimization using the graph cartesian product. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019. arXiv:1902.00448.

Bernt Øksendal. *Stochastic Differential Equations: An Introduction with Applications*. Universitext. Springer, Berlin, Heidelberg, 6 edition, 2003.

Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.

Christian P Robert and George Casella. *Monte Carlo statistical methods*, volume 2. Springer, 1999.

Binxin Ru, Ahsan Alvi, Vu Nguyen, Michael A Osborne, and Stephen Roberts. Bayesian optimisation over multiple continuous and categorical inputs. In *International Conference on Machine Learning (ICML)*, pages 8276–8285. PMLR, 2020. arXiv:1906.08878.

Reuven Y Rubinstein and Dirk P Kroese. *Simulation and the Monte Carlo method*. John Wiley & Sons, 2016.

Amar Shah, Andrew G Wilson, and Zoubin Ghahramani. Bayesian optimization using student-t processes. In *NIPS Workshop on Bayesian Optimization*, 2013.

Michael Stein. Large sample properties of simulations using latin hypercube sampling. *Technometrics*, 29(2):143–151, 1987.

S Surjanovic and D Bingham. Virtual library of simulation experiments: Test functions and datasets. Retrieved August 25, 2024, from http://www.sfu.ca/~ssurjano, 2024.

Xingchen Wan, Vu Nguyen, Huong Ha, Binxin Ru, Cong Lu, and Michael A Osborne. Think global and act local: Bayesian optimisation over high-dimensional categorical and mixed search spaces. In *International Conference on Machine Learning (ICML)*, pages 10663–10674. PMLR, 2021a. arXiv:2102.07188.

Xingchen Wan, Vu Nguyen, Huong Ha, Binxin Ru, Cong Lu, and Michael A Osborne. Think global and act local: Bayesian optimisation over high-dimensional categorical and mixed search spaces. In *International Conference on Machine Learning (ICML)*, pages 10663–10674. PMLR, 2021b.

Jie Wang. An intuitive tutorial to gaussian processes regression. *arXiv preprint arXiv:2009.10862*, 2020.

Xiaoxing Wang, Jiaxing Li, Chao Xue, Wei Liu, Chaoyue Wang, Weifeng Liu, Xiaokang Yang, Junchi Yan, and Dacheng Tao. Poisson process for bayesian optimization. *arXiv preprint arXiv:2205.16096*, 2022.

Xilu Wang, Yaochu Jin, Sebastian Schmitt, and Markus Olhofer. Recent advances in bayesian optimization. *ACM Computing Surveys*, 55(13s): 1–36, 2023.

Shuhei Watanabe. Tree-structured parzen estimator: Understanding its algorithm components and their roles for better empirical performance. *arXiv preprint arXiv:2304.11127*, 2023.

Xueli Xiao, Ming Yan, Sunitha Basodi, Chunyan Ji, and Yi Pan. Efficient hyperparameter optimization in deep learning using a variable length genetic algorithm. *arXiv preprint arXiv:2006.12703*, 2020.

Yichi Zhang, Siyu Tao, Wei Chen, and Daniel W Apley. A latent variable approach to gaussian process modeling with qualitative and quantitative factors. *Technometrics*, 62(3):291–302, 2020.

Gordan Žitković. Introduction to stochastic processes-lecture notes. *Department of Mathematics, The University of Texas at Austin*, 2010.